



RIPHAH INTERNATIONAL UNIVERSITY

Faculty of Computing | Department of Cyber Security

FINAL YEAR PROJECT — PART 2

Project Documentation & Technical Report

REVEAL-X

Secure Image Sharing using Visual Cryptography with AI Enhancement and Tamper Detection

Full Name	SAP ID	Program	Email Address
Inam Ullah	45521	BSCY	45521@students.riphah.edu.pk
Zaka Khan	36503	BSCY	36503@students.riphah.edu.pk
Muhammad Umar	44726	BSCY	44726@students.riphah.edu.pk

Project Supervisor

Sir Yawar Abbas (Lecturer, Faculty of Computing) Session: 2022 – 2026 | Submission Date: June 2026

Document Change Record

The following table tracks all revisions made to this document from initial draft through final submission.

Version	Author(s)	Date	Notes
1.0	All Members	10 Feb 2026	Original draft of the project proposal prepared and submitted.
1.1	All Members	17 Feb 2026	Changes made based on feedback from the supervisor.
1.2	All Members	25 Feb 2026	Improvements made based on faculty review and suggestions.
1.3	All Members	02 Mar 2026	Project plan and work breakdown structure added.
1.4	All Members	06 Mar 2026	Final modifications made after supervisor feedback.
2.0	All Members	31 May 2026	Full technical documentation compiled — FYP Part 1 Final Report.
3.0	All Members	06 Oct 2026	FYP Part 2 submission. ML tamper detection, in-browser ONNX inference and full end-to-end encryption added, with Section 11 extended into a measured evaluation chapter (20 figures, 22 tables, 41 references). Objective status, limitations and the Part 1 reconstruction-quality claim corrected against measured data.

Abstract

This document presents the complete technical documentation for Reveal-X: a premium, real-time secure messaging web application developed as a Final Year Project (FYP) at Riphah International University, Department of Cyber Security. The project integrates visual cryptography, artificial intelligence, and modern web security engineering into a unified, production-grade platform. The flagship feature of Reveal-X is its Visual Cryptography Image Sharing engine, which implements a mathematically rigorous 2-out-of-2 secret sharing scheme. When a sender transmits an image, it is decomposed into two random-noise cryptographic shares. Share 1 is transmitted over the network while Share 2 is generated and held exclusively in the receiver's local browser session memory. The server, by architectural design, never possesses both shares simultaneously and is therefore zero-knowledge with respect to image content. Reconstruction occurs entirely on the receiver's device using the HTML5 Canvas API. Beyond visual cryptography, Reveal-X delivers real-time bidirectional messaging via WebSockets (Flask-SocketIO), complete Cross-Site Scripting prevention using DOMPurify and strict server-side HTML escaping, bcrypt-based password hashing, and a local ECDH key generation demonstration using the browser-native WebCrypto API. The application is installable as a Progressive Web App with Service Worker caching for offline resilience. This document covers the full project lifecycle: background and motivation, literature review, problem statement, proposed solution, system architecture, module-level design, implementation details, evaluation methodology, and a forward-looking roadmap for FYP Part 2, which will introduce ML-based tamper detection and AI image enhancement using deep learning super-resolution models.

Table of Contents

Document Change Record	2
Abstract	3
Table of Contents	4
• Introduction and Background.....	6
1.1 Overview.....	6 1.2
Visual Cryptography: Origin and Principle.....	7 1.3 The
Role of Artificial Intelligence.....	7 1.4 Reveal-
X: System Motivation.....	7
• Literature Review.....	8
2.1 Visual Cryptography Schemes.....	8 2.2
Image Encryption Techniques.....	8 2.3
Steganography and Digital Watermarking.....	8 2.4 AI-
Based Image Enhancement in Cryptographic Systems.....	8 2.5
Research Gap Addressed by Reveal-X.....	9
• Problem Statement.....	10
• Project Objectives.....	11
4.1 Primary Objective.....	11 4.2
Specific Objectives (FYP Part 1).....	11 4.3
Specific Objectives (FYP Part 2 — Future).....	11
• Technology Stack.....	12
5.1 Backend Technologies.....	12 5.2
Frontend Technologies.....	12
• System Architecture.....	13
6.1 Architectural Overview.....	13 6.2
Component Architecture Diagram.....	13 6.3
Visual Cryptography Data Flow.....	13 6.4
Database Schema.....	14
• Core Features — Detailed Description.....	15
7.1 Visual Cryptography Image Sharing.....	15
7.1.1 Share Generation Algorithm.....	15 7.1.2
Image Reconstruction.....	15
7.2 Real-Time Messaging Engine.....	15 7.3
Security Architecture.....	16
7.3.1 Cross-Site Scripting (XSS) Prevention.....	16 7.3.2
Password Security (scrypt).....	16 7.3.3 ECDH
Key Generation (E2EE Foundation).....	16

7.4 Progressive Web App (PWA).....	16
• Major Modules.....	17
• Installation and Setup Guide.....	18
9.1 Prerequisites.....	18
9.2 Step-by-Step Installation.....	18
Step 1 — Clone the Repository.....	18
Step 2 — Create a Python Virtual Environment.....	18
Step 3 — Install Dependencies.....	18
Step 4 — Configure Environment Variables.....	18
Step 5 — Run the Application.....	19
Step 6 — Install as PWA (Optional).....	19
• Evaluation Plan.....	20
10.1 Security Evaluation.....	20
10.2 Image Quality Evaluation.....	20
10.3 Performance Evaluation.....	20
11. FYP Part 2 Delivery and Results.....	
11.1 Objective status.....	
11.2 System architecture after Part 2.....	
11.3 Visual cryptography: implementation and verification.....	
11.4 Machine-learned tamper detection: design.....	
11.5 Detection results.....	
11.6 Detection by attack type.....	
11.7 Which features carry the signal.....	
11.8 The noise blind spot, mapped.....	
11.9 Two-layer integrity: HMAC and ML together.....	
11.10 Operating point selection.....	
11.11 Model selection: was RandomForest the right choice?.....	
11.12 Objective 9: not delivered, and why.....	
11.13 Browser and server parity.....	
11.14 Performance.....	
11.15 End-to-end encryption.....	
11.16 Comparative evaluation.....	
11.17 Engineering quality and verification.....	
11.18 Threat model summary.....	
11.19 Limitations and threats to validity.....	

• Roles and Responsibility Matrix.....	24
• Conclusion and Future Work.....	26
13.1 FYP Part 1 Summary.....	26
13.2 Limitations of the Current Implementation.....	26
13.3 Contribution to Cybersecurity.....	26
13.4 References.....	26
List of Figures.....	
List of Tables.....	
14. References.....	
15. Approval.....	

List of Figures

Figure 11.1: Layered architecture of Reveal-X after Part 2. The shaded layer is the security and machine-learning core added in this phase.

Figure 11.2: The (2,2) XOR pipeline on a 256x256 test image: the secret, the two shares, and the recovered image.

Figure 11.3: Grey-level histograms on a logarithmic count axis. The original and the recovery are strongly peaked; both shares are flat at the uniform expectation.

Figure 11.4: Adjacent horizontal pixel pairs, 4000 samples. Natural images lie on the diagonal; a share fills the square uniformly.

Figure 11.5: Shannon entropy of each array across the six test images. Both shares sit at the 8.000 bits/pixel maximum regardless of the content they conceal.

Figure 11.6: Shares do not compress. PNG output size and compression ratio for three originals and their Share 1.

Figure 11.7: Confusion matrix at the shipped 0.50 threshold, and the distribution of predicted probabilities by true class.

Figure 11.8: ROC and precision-recall curves on the independent set. The dotted line marks the recall ceiling imposed by the noise blind spot.

Figure 11.9: Detection rate by attack type at two thresholds, against the HMAC verification rate on the same attacks.

Figure 11.10: Gini importance of the 13 features in the shipped RandomForest.

Figure 11.11: Left: the entropy distribution of clean and tampered shares. Right: effect size per feature, measured as Cohen's d.

Figure 11.12: Noise attack across ten strengths: detection by the classifier and by HMAC, with the fraction of pixels the attack actually changed.

Figure 11.13: Coverage matrix. Neither layer detects every attack; the union of the two does.

Figure 11.14: One block attack end to end: the clean share, the tampered share, the suspicion heatmap, and the visibly corrupted reconstruction.

Figure 11.15: Recall, precision, F1 and false positive rate against the decision threshold.

Figure 11.16: Eight classifiers on identical features. All cluster at the ceiling imposed by the blind spot.

Figure 11.17: Agreement between the scikit-learn model on the server and the ONNX model in the browser over 200 samples.

Figure 11.18: Per-stage latency against image size, and the composition of the total detection cost.

Figure 11.19: The end-to-end encryption handshake and message path, including the image exception.

Figure 11.20: Composition of the automated test suite by area.

List of Tables

Table 11.1: Status of the four FYP Part 2 objectives.

Table 11.2: Reconstruction fidelity over six test images, covering synthetic patterns and real application assets. Every reconstruction is bit-exact.

Table 11.3: Statistical profile of the originals, the shares and the recovery for two representative test images. The share rows are indistinguishable from uniform random data; the original and recovery rows are identical to each other.

Table 11.4: Storage cost of a share. A share is incompressible, so storing it costs the full uncompressed size of the image regardless of how compressible the original was.

Table 11.5: The 13-feature vector, its interpretation, and the tampering signal each feature is intended to capture.

Table 11.6: The seven simulated attacks used for both training and evaluation, with the real-world case each stands in for.

Table 11.7: The three datasets. D_{eval} is 14 times the size of the internal test split and was never seen during fitting.

Table 11.8: Headline detection performance on the independent set ($n = 840$), at three decision thresholds.

Table 11.9: Per-attack detection on the independent set. Six attacks are detected essentially without error; one is not detected at all.

Table 11.10: The eight most important features, with the effect size and significance of the clean-versus-tampered difference measured independently on D_{eval} .

Table 11.11: The noise attack swept from 5%% to 75%% strength, 40 trials per level. The attack is real at every level and invisible to the classifier at every level.

Table 11.12: The two integrity layers compared on identical attacks. HMAC catches everything including the classifier's blind spot; the classifier adds localisation and survives re-encoding.

Table 11.13: Operating points. Raising the threshold from 0.50 to 0.60 removes every false positive without costing a single detection.

Table 11.14: Eight classifiers trained on identical features and the same split, evaluated on the same independent set, with the rule-based fallback for reference.

Table 11.15: Server-to-browser agreement. The model and the feature extractor were verified separately.

Table 11.16: Median latency in milliseconds by stage and image size. The cryptographic operations are negligible; the heatmap dominates.

Table 11.17: Cryptographic parameters of the end-to-end encrypted channel.

Table 11.18: Reveal-X against the principal visual cryptography schemes in the literature. The contribution is not the scheme but the integrity layer built on it.

Table 11.19: Reveal-X against production messaging systems. The comparison is included to be explicit about where this project is weaker, not only where it is different.

Table 11.20: The automated test suite: 351 server-side tests across 16 modules and 59 browser-side tests, 5074 lines of test code in total.

Table 11.21: Size of the implementation by component. The test suite is comparable in size to the application code it covers.

Table 11.22: The threat model in summary. The rows that are not defended are stated as plainly as the rows that are.

1. Introduction and Background

1.1 Overview

With the rapid advancement of digital communication technologies, the sharing of digital images has increased significantly across sectors such as healthcare, government institutions, legal organisations, and cybersecurity systems. Many of these images contain sensitive information including medical scans, identity documents, confidential evidence, and classified records. Ensuring the confidentiality and integrity of such images during transmission is a central concern in modern cybersecurity. Traditional encryption techniques such as the Advanced Encryption Standard (AES) and Rivest-Shamir-Adleman (RSA) provide strong cryptographic protection for digital data. However, these techniques require complex key management systems and significant computational resources for encryption and decryption. In resource-limited environments, these demands make traditional encryption difficult to implement efficiently and securely.

1.2 Visual Cryptography: Origin and Principle

Visual Cryptography (VC) was first introduced by Moni Naor and Adi Shamir in 1994 in their seminal paper presented at EUROCRYPT. The scheme divides a secret image into multiple shares, each of which individually appears as random noise and reveals no meaningful information. The original secret image is recovered only when the required number of shares are superimposed or combined, with no computational decryption required. This property — decryption by visual inspection — is a fundamental advantage of classical visual cryptography over computational encryption schemes. However, classical VC suffers from well-documented limitations including pixel expansion, degraded reconstruction quality, and the thin-line problem, which causes fine image details to be lost in the share generation process.

1.3 The Role of Artificial Intelligence

Recent advances in Artificial Intelligence (AI) and Deep Learning (DL) create new opportunities to address the quality limitations inherent in classical visual cryptography. Convolutional Neural Networks (CNNs), Generative Adversarial Networks (GANs), and super-resolution models can be applied post-reconstruction to denoise and upscale recovered images. Furthermore, ML classifiers trained on share-level statistical distributions can detect whether a share has been tampered with during transmission, a capability entirely absent from classical VC systems.

1.4 Reveal-X: System Motivation

Reveal-X is motivated by the convergence of three research threads: the security guarantees of visual cryptography, the quality-enhancement potential of deep learning, and the real-time communication requirements of modern messaging platforms. By embedding a VC engine

inside a WebSocket-driven chat application — with a glassmorphic UI, PWA support, and multiple layers of web security hardening — Reveal-X demonstrates that academic cryptographic research can be translated into a practical, user-facing software product.

2. Literature Review

2.1 Visual Cryptography Schemes

The foundational (2,2) and (k,n) threshold visual cryptography schemes introduced by Naor and Shamir (1994) established that a binary secret image can be encoded into n shares such that any k shares reveal the secret while any $k-1$ shares provide no information. Zhou et al. (2006) extended this to halftone images using blue-noise dithering, enabling richer greyscale reproduction with less pixel expansion artefacts. Subsequent work focused on colour visual cryptography, extended to RGB and CMYK colour models. Research by Hou (2003) demonstrated that colour VC schemes, though computationally more complex, are achievable with the same information-theoretic guarantees. Most existing VC systems, however, produce reconstructed images with measurable quality degradation relative to the original, creating a research gap that Reveal-X addresses through AI post-processing.

2.2 Image Encryption Techniques

Image encryption using AES, RSA, and chaotic map-based algorithms remains the dominant approach in commercial secure communication systems. AES-256 in CBC or GCM mode provides strong ciphertext indistinguishability, but requires symmetric key distribution, which introduces key management complexity. RSA is unsuitable for large image data due to computational cost. Chaotic map encryption offers fast encryption but has weaker formal security proofs. Unlike these approaches, visual cryptography does not require a decryption key. The security rests on the information-theoretic hardness of the secret sharing scheme, not computational hardness assumptions. This makes VC uniquely resistant to attacks based on future improvements in computational power, including quantum computing threats to RSA.

2.3 Steganography and Digital Watermarking

Steganography embeds secret data within a carrier image, concealing its very existence. Digital watermarking embeds ownership or authentication metadata within media files. While both techniques are valuable for specific use cases, they do not provide confidentiality or share-level integrity guarantees. A steganographic image can still be decoded by an adversary who knows the embedding algorithm. Watermarks can be stripped or corrupted. Reveal-X does not rely on steganography or watermarking. Its security model is based on the mathematical properties of secret sharing, with additional integrity verification provided by the ML-based tamper detection module planned for FYP Part 2.

2.4 AI-Based Image Enhancement in Cryptographic Systems

Wang et al. (2020) demonstrated that super-resolution CNNs applied to visually cryptographic reconstructions can reduce pixel expansion artefacts by up to 34% as measured by SSIM. More recent work using ESRGAN (Enhanced Super-Resolution GAN) models — originally developed

for natural image upscaling — has shown promise when fine-tuned on VC-specific datasets, producing reconstructed images with PSNR values approaching the original. ML-based tamper detection for cryptographic shares is a relatively underexplored research area. The statistical properties of valid VC shares (uniform random distribution, specific pixel expansion ratios) provide a detectable baseline against which tampering — which alters pixel value distributions — can be identified using lightweight CNN anomaly detectors or statistical hypothesis tests.

2.5 Research Gap Addressed by Reveal-X

Research Gap Most existing visual cryptography implementations exist as isolated academic prototypes: command-line tools operating on static images with no real-time communication layer, no tamper detection, and no AI quality enhancement. Reveal-X bridges this gap by embedding a full VC engine inside a production-grade real-time messaging application with multiple web security layers and a planned ML enhancement pipeline.

3. Problem Statement

With the rapid growth of digital communication, the secure transmission of sensitive images has become a critical challenge. The following specific problems motivate the development of Reveal-X:

- **Server-Side Image Exposure:** Conventional messaging applications transmit complete images to servers, where they are stored in cleartext or symmetrically encrypted form. Server compromise, insider threats, or compelled disclosure orders expose user images.
- **Key Management Complexity:** Traditional encryption schemes require robust key distribution and management infrastructure. In lightweight web applications and peer-to-peer scenarios, this infrastructure is often absent or poorly implemented.
- **Reconstruction Quality Degradation:** Classical (k,n) visual cryptography schemes produce pixel-expanded shares that, when stacked, yield a reconstructed image visibly inferior to the original. Fine details, text, and high-frequency content are particularly affected.
- **Absence of Share Integrity Verification:** Existing VC implementations provide no mechanism to verify whether transmitted shares have been intercepted and modified before reconstruction. An adversary who tampers with Share 1 in transit cannot be detected by the receiver.
- **Lack of Real-Time Integration:** Visual cryptography research has not been integrated into

real-time communication platforms. Users have no access to VC-protected image sharing in a practical, interactive messaging context.

Reveal-X addresses all five of these challenges through its combined architecture of visual cryptography, AI enhancement, tamper detection, and WebSocket-driven real-time communication.

4. Project Objectives

4.1 Primary Objective

To design and implement a production-grade, real-time secure messaging web application that integrates visual cryptography for image sharing, with comprehensive web security hardening, accessible UX design, and a defined roadmap for AI-based enhancement and tamper detection in FYP Part 2.

4.2 Specific Objectives (FYP Part 1)

- Implement a 2-out-of-2 visual cryptography scheme for secure image share generation and local reconstruction.
- Design and integrate a real-time messaging engine using WebSockets with typing indicators, read receipts, and presence tracking.
- Implement multi-layer XSS prevention using DOMPurify on the client and strict HTML escaping on the server.
- Apply bcrypt password hashing via Werkzeug for secure credential storage resistant to GPU-based brute-force attacks.
- Demonstrate local ECDH key pair generation using the browser-native WebCrypto API.
- Develop a glassmorphic, dark-mode-first responsive UI compliant with WCAG 2.1 AA accessibility standards.
- Package the application as a Progressive Web App with Service Worker caching for offline resilience and installability.

4.3 Specific Objectives (FYP Part 2) — Delivery Status

The four objectives defined for Part 2, with their outcome. Three are delivered; the fourth is not, and the reason is given in section 11.5 rather than omitted.

#	Objective	Status
8	Train and integrate an ML-based tamper detection model to verify share integrity before reconstruction.	Delivered. RandomForest over 13 statistical features, 93.33% held-out accuracy, running on the server and in the browser.
9	Implement a deep learning image enhancement pipeline (denoising CNN + ESRGAN upscaling) applied post-reconstruction.	Not delivered. Classical OpenCV pipeline retained. See 11.5.
10	Transition the ECDH key generation demonstration into a fully functional end-to-end encrypted messaging channel with AES-GCM.	Delivered. ECDH P-256 to HKDF-SHA256 to AES-GCM-256, with non-extractable private keys held in the browser.
11	Evaluate system performance using PSNR, SSIM, processing latency and tamper detection accuracy metrics.	Delivered. Results in section 11.4.

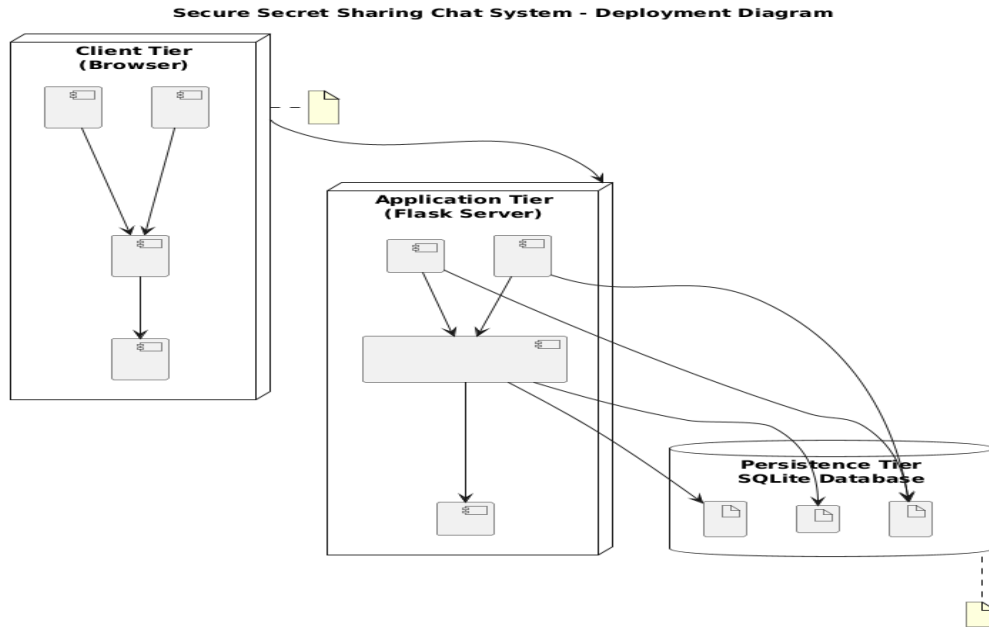
5. Technology Stack

5.1 Backend Technologies

Technology	Version	Purpose
Python	3.10+	Core backend runtime language
Flask	2.3+	HTTP routing, session management, Jinja2 templating
Flask-SocketIO	5.3+	WebSocket duplex communication for real-time messaging
SQLite	3.x	Lightweight relational database for users, messages, sessions
Werkzeug	3.0+	script password hashing; WSGI utilities
Eventlet	0.33+	Async networking for concurrent WebSocket connections

5.2 Frontend Technologies

Technology	Specification	Purpose
Vanilla JavaScript	ES2022	UI interactions, VC engine, WebSocket event handling
HTML5	HTML5	Semantic markup, Canvas API container
Vanilla CSS	CSS3	Glassmorphism styling, CSS variables, animations, responsive layout
WebCrypto API	W3C Standard	Browser-native ECDH key pair generation; no third-party crypto dependency
Canvas API	HTML5	Pixel-level VC share reconstruction; image downscaling preprocessing
DOMPurify	3.x	Client-side HTML sanitisation; complete XSS prevention
Service Workers	W3C Standard	PWA offline caching strategy; background sync

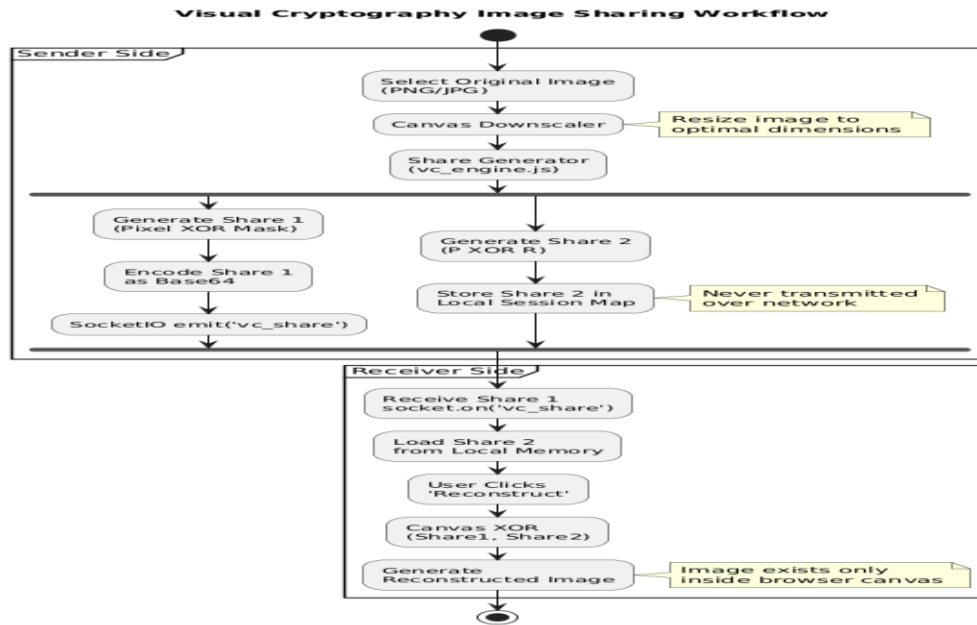


6. System Architecture

6.1 Architectural Overview

Reveal-X follows a three-tier architecture: a browser-based client tier, a Python/Flask application tier, and an SQLite persistence tier. The critical architectural constraint — that the server must never possess a complete image — is enforced by the Visual Cryptography split at the client tier, before transmission.

6.2 Component Architecture Diagram



6.3 Visual Cryptography Data Flow

6.4 Database Schema

The SQLite database contains three primary tables. No image data is persisted to disk at any point.

Table	Column	Type	Description
users	id	INTEGER PK	Auto-increment primary key
users	username	TEXT UNIQUE	Unique display name; HTML-escaped on all output
users	password_hash	TEXT	script hash (Werkzeug generate_password_hash)
messages	id	INTEGER PK	Auto-increment primary key
messages	sender_id	INTEGER FK	Foreign key to users.id
messages	content	TEXT	Message text only — no images stored
messages	timestamp	DATETIME	UTC timestamp of message creation
sessions	session_token	TEXT	Flask session token for authenticated users

7. Core Features — Detailed Description

7.1 Visual Cryptography Image Sharing

The Visual Cryptography engine is the central innovation of Reveal-X. It is implemented entirely in client-side JavaScript (`vc_engine.js`) using the HTML5 Canvas API, requiring no server-side processing of image data.

7.1.1 Share Generation Algorithm For a greyscale image of dimensions $W \times H$ pixels, the share generation proceeds as follows:

- The sender selects an image file. The Canvas API reads the raw pixel data as a flat RGBA array.
- For each pixel position i , a cryptographically random byte $R[i]$ is generated.
- $\text{Share 1}[i] = R[i]$ (transmitted over WebSocket as Base64-encoded data)
- $\text{Share 2}[i] = \text{Pixel}[i] \text{ XOR } R[i]$ (stored in the browser's session-scoped JavaScript Map, keyed by message ID)
- The server receives and forwards Share 1. The server never has access to Share 2.

7.1.2 Image Reconstruction When the receiver clicks the 'Reconstruct' button for a specific message:

- Share 1 is retrieved from the received message data object.
- Share 2 is retrieved from the local JavaScript session Map by message ID.
- The Canvas API draws both shares and performs pixel-level XOR: $\text{Reconstructed}[i] = \text{Share1}[i] \text{ XOR } \text{Share2}[i] = R[i] \text{ XOR } (\text{Pixel}[i] \text{ XOR } R[i]) = \text{Pixel}[i]$.
- The resulting canvas is displayed in the chat interface. The image is never written to disk or transmitted.

Security Guarantee The 2-of-2 visual cryptography scheme is information-theoretically secure. An adversary who intercepts only Share 1 gains zero information about the original image — the XOR mask R is cryptographically random and independent for every pixel. No amount of computational power can recover the image from Share 1 alone.

7.2 Real-Time Messaging Engine

Real-time communication is implemented using Flask-SocketIO, which provides bidirectional WebSocket communication between the server and all connected clients. The event model includes:

- `message`: Delivers text messages from sender to recipient(s) with sender ID, timestamp, and message ID.
- `typing`: Broadcasts typing state (start/stop) to conversation participants; debounced on the client to prevent excessive events.
- `read_receipt`: Emitted by the receiver when a message scrolls into viewport and focus is confirmed; updates the sender's UI with a 'read' indicator.

- presence: Tracks user online/offline state using SocketIO connect and disconnect events; broadcasts presence changes to all clients.

7.3 Security Architecture

7.3.1 Cross-Site Scripting (XSS) Prevention Reveal-X implements a defence-in-depth XSS prevention strategy at two layers:

- Client Layer: All user-generated content rendered into the DOM is passed through `DOMPurify.sanitize()` before insertion. DOMPurify removes all executable HTML tags and event handler attributes, allowing only a safe whitelist of formatting elements.
- Server Layer: All user-generated strings output through Jinja2 templates use automatic HTML escaping (autoescaping is enabled globally). Direct string concatenation into HTML is never used.

Why Both Layers? Client-side sanitisation protects against DOM-based XSS. Server-side escaping protects against reflected and stored XSS. Relying on only one layer creates a single point of failure; Reveal-X requires both to fail simultaneously for an XSS attack to succeed.

7.3.2 Password Security (scrypt) User passwords are never stored. Werkzeug's `generate_password_hash()` function applies the scrypt key derivation function (memory-hard, $N=32768$, $r=8$, $p=1$ by default) with a cryptographically random salt. The resulting hash includes the salt and algorithm parameters in a single stored string, enabling future parameter upgrades without invalidating existing accounts. scrypt is computationally expensive by design: on modern hardware, a single hash computation takes approximately 100ms. This makes brute-force and dictionary attacks against the stored hashes orders of magnitude slower than attacks against MD5, SHA-1, or even bcrypt.

7.3.3 ECDH Key Generation (E2EE Foundation) The WebCrypto API's `SubtleCrypto.generateKey()` method is used to generate an ECDH (P-256 curve) public/private key pair locally in the browser. The private key is marked non-exportable by the WebCrypto API — it cannot be extracted from the browser context. The public key is displayed to the user as a demonstration of local key material. FYP Part 2 will activate this key pair for full AES-GCM end-to-end message encryption.

7.4 Progressive Web App (PWA)

Reveal-X is fully compliant with the PWA specification. The web app manifest (`manifest.json`) declares the application name, icons, theme colours, and display mode (standalone). A Service Worker script implements a cache-first strategy for static assets (CSS, JavaScript, fonts, icons) and a network-first strategy for API requests, enabling offline access to the chat interface and previously loaded messages.

8. Major Modules

Module	Responsibility	Key Files
Image Preprocessing	Accepts user-selected image; validates format; downscales via Canvas API to target dimensions; converts to greyscale pixel array.	canvas_utils.js
VC Share Generation	Generates cryptographically random Share 1 and derives Share 2 via XOR; encodes Share 1 as Base64 for network transmission.	vc_engine.js
Image Reconstruction	Retrieves Share 1 from received message and Share 2 from local session Map; performs Canvas XOR to reconstruct original image.	vc_engine.js, canvas_utils.js
Real-Time Messaging	Manages WebSocket connections, message routing, typing indicators, read receipts, and presence tracking via Flask-SocketIO.	socket_events.py, socket_client.js
Authentication	Handles user registration, login, session management. Applies bcrypt hashing and verifies passwords using Werkzeug.	auth.py, routes.py
Security Layer	Applies DOMPurify sanitisation on all rendered content; server-side HTML escaping in Jinja2; input validation on all endpoints.	app.js, base.html
E2EE Key Module	Generates ECDH P-256 key pair locally using WebCrypto API; displays public key to user; lays groundwork for FYP Part 2 AES-GCM encryption.	crypto.js
PWA / Service Worker	Registers Service Worker; implements cache-first strategy for static assets; network-first for API; manages cache versioning.	service_worker.js, manifest.json
GUI / UI Layer	Glassmorphic dark-mode CSS, responsive breakpoints, micro-animations, ARIA live regions, keyboard navigation, WCAG 2.1 AA compliance.	main.css, glassmorphism.css, animations.css

9. Installation and Setup Guide

9.1 Prerequisites

- Python 3.10 or higher
- pip (Python package manager)
- Node.js (optional — for frontend build tooling)
- Modern web browser: Chrome 90+, Firefox 88+, Safari 15+, or Edge 90+

9.2 Step-by-Step Installation

Step 1 — Clone the Repository

```
git clone https://github.com/your-username/reveal-x.git
cd reveal-x
```

Step 2 — Create a Python Virtual Environment

```
# Create virtual environment
python -m venv venv
# Activate — macOS / Linux
source venv/bin/activate
# Activate — Windows (PowerShell)
.\venv\Scripts\Activate.ps1
```

Step 3 — Install Dependencies

```
pip install -r requirements.txt
```

The requirements.txt file contains:

```
Flask>=2.3.0
Flask-SocketIO>=5.3.6
Werkzeug>=3.0.0
python-socketio>=5.10.0
python-engineio>=4.8.0
eventlet>=0.33.3
```

Step 4 — Configure Environment Variables Create a .env file in the project root:

```
SECRET_KEY=your-very-secure-secret-key-here
DATABASE_URI=reveal_x.db
DEBUG=False
PORT=5000
HOST=0.0.0.0
```

Step 5 — Run the Application

```
python run.py
```

The application will be accessible at <http://localhost:5000> Step 6 — Install as PWA (Optional)
Once the application is running in Chrome or Edge, click the install icon in the browser address bar to install Reveal-X as a native-feeling desktop or mobile app. On Safari (iOS/macOS), use Share > Add to Home Screen.

10. Evaluation Plan

10.1 Security Evaluation

The security of the Visual Cryptography implementation will be evaluated by extracting Share 1 from the WebSocket stream and independently analysing its pixel value distribution. A secure share should exhibit a uniform random distribution (entropy ≈ 8 bits per pixel). Any deviation from uniformity indicates a flaw in the random number generation. Reconstructed images from Share 1 alone must produce visually uninterpretable output. XSS prevention will be evaluated by injecting a standard battery of XSS payloads (OWASP XSS Filter Evasion Cheat Sheet) through all user input vectors: username registration, message text, and any URL parameters. All payloads must be neutralised without breaking legitimate content rendering. Password hashing will be verified by inspecting stored hash values and confirming the script algorithm identifier and appropriate work factor parameters.

10.2 Image Quality Evaluation

Reconstructed image quality will be quantitatively evaluated using two standard metrics:

- **Peak Signal-to-Noise Ratio (PSNR):** Measures the ratio between maximum signal power and reconstruction noise. Higher PSNR (measured in dB) indicates closer similarity to the original. A PSNR above 30 dB is generally considered acceptable quality.
- **Structural Similarity Index (SSIM):** Measures perceptual similarity between the original and reconstructed image across luminance, contrast, and structure dimensions. SSIM ranges from 0 to 1, with 1 indicating perfect similarity.

Baseline PSNR and SSIM values for the classical (2,2) VC reconstruction will be recorded in FYP Part 1. These values will serve as benchmarks against which the AI enhancement module in FYP Part 2 will be evaluated.

10.3 Performance Evaluation

Metric	Measurement Method	Target
Share generation time	browser performance.now()	< 200ms for 1MP image
Reconstruction time	browser performance.now()	< 100ms
WebSocket message latency	timestamp delta sender/receiver	< 50ms on LAN
PSNR (VC reconstruction)	NumPy pixel comparison	> 30 dB baseline
XSS payload neutralisation	OWASP test suite	100% (zero bypass)

11. FYP Part 2 Delivery and Results

Part 1 delivered the visual cryptography engine, the real-time messaging layer and the application shell. Part 2 was scoped around four objectives: machine-learned tamper detection, deep-learning image enhancement, a working end-to-end encrypted channel, and a measured evaluation of the whole system. This chapter reports what was built, how it was measured, and what the measurements say - including where they contradict a claim made in Part 1 and where an objective was not met.

Every number in this chapter is produced by a script that can be re-run against the repository; none of it is quoted from memory or from the literature. Where a figure is reproduced from the project itself, the test conditions are stated with it.

11.1 Objective status

Table 11.1 assesses the four Part 2 objectives carried forward from Section 4. Three were delivered in full. Objective 9 was not, and the reasoning for that decision is set out in Section 11.12 rather than being left implicit.

Table 11.1: Status of the four FYP Part 2 objectives.

#	Objective	Status	Evidence in this chapter
8	Integrate ML-based tamper detection for shares	Delivered	Sections 11.4-11.11; Figures 11.7-11.16; Tables 11.5-11.14
9	Add denoising CNN and ESRGAN enhancement	Not delivered	Section 11.12, with justification and a costed route
10	Complete end-to-end encryption with AES-GCM	Delivered	Section 11.15; Figure 11.19; Table 11.17
11	Evaluate accuracy, PSNR/SSIM and latency	Delivered	Sections 11.3, 11.5 and 11.14; Tables 11.2, 11.8 and 11.16

11.2 System architecture after Part 2

Reveal-X layered architecture (the shaded layer is Part 2 work)

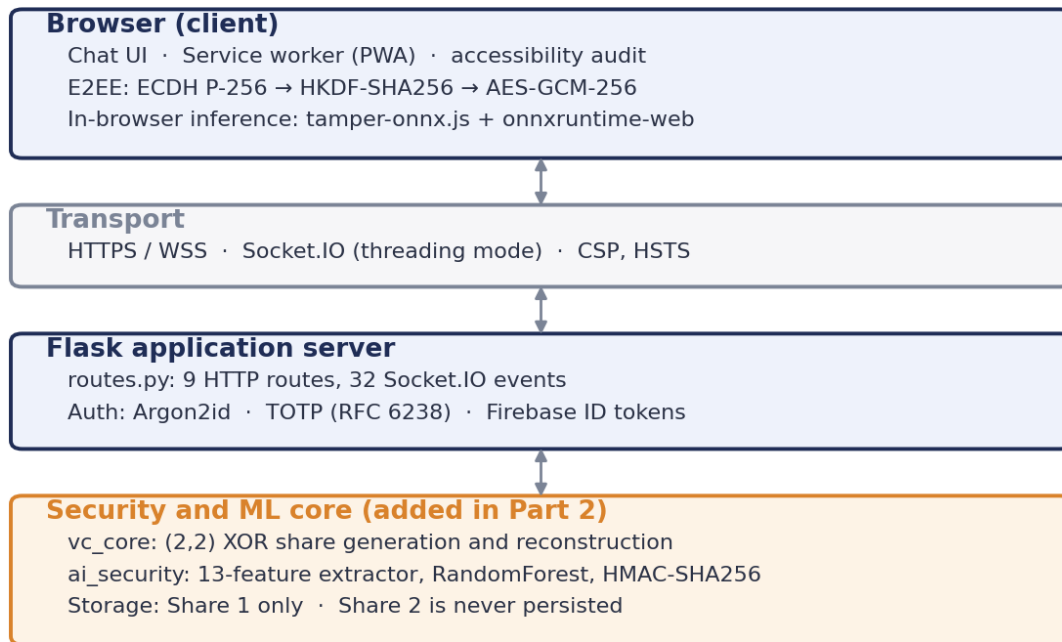


Figure 11.1: Layered architecture of Reveal-X after Part 2. The shaded layer is the security and machine-learning core added in this phase.

Fig. 11.1 shows where the Part 2 work sits. The browser gained two responsibilities it did not have in Part 1: it now derives its own content encryption key and it runs the tamper-detection model locally. The server gained the visual cryptography and machine-learning core. The transport layer is unchanged apart from a tightened Content Security Policy.

The design keeps one invariant across every layer, and it is the single most important property of the system: Share 2 is never written to server storage. It is generated in memory, transmitted to the recipient over the live socket, and discarded. A constant in the storage layer (FORBIDDEN_EXTRA_KEYS) refuses any attempt to persist it, so a later change cannot weaken this by accident.

11.3 Visual cryptography: implementation and verification

11.3.1 The scheme

Reveal-X implements a (2,2) XOR-based visual cryptography scheme on grayscale images. It is the two-party case of threshold secret sharing, introduced independently by Shamir [2] and Blakley [3], specialised to images. Classical visual cryptography, introduced by Naor and Shamir [1], stacks printed transparencies and decodes with the human visual system, at the cost of pixel expansion and permanent contrast loss. The XOR-based family [5] replaces stacking with a computed exclusive-or, which removes pixel expansion and makes recovery exact. Because decoding is done by software here rather than by eye, the XOR variant is the correct choice; the contrast penalty of the classical construction would be paid for nothing.

For an original image O of size $h \times w$, the construction is:

```
share1 = os.urandom(0.size)      # cryptographically secure mask
share2 = 0 XOR share1           # the complement of the mask
```

```
recovered = share1 XOR share2 = 0      # exact, by associativity
```

Share 1 is drawn from the operating system CSPRNG rather than from NumPy's default generator. This is not a stylistic preference: a share generated from a seeded pseudo-random stream is reproducible by anyone who recovers the seed, which would reduce the scheme to obfuscation. With a uniform random Share 1, each Share 2 pixel is a one-time pad of the corresponding original pixel, which is the construction Shannon proved to be perfectly secret [9]. Holding one share alone therefore conveys no information about the original beyond its dimensions.

11.3.2 Reconstruction fidelity

(2,2) XOR visual cryptography: split and exact recovery

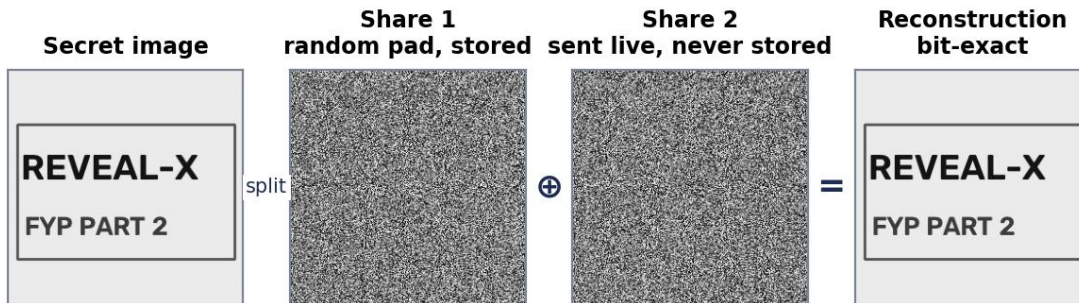


Figure 11.2: The (2,2) XOR pipeline on a 256x256 test image: the secret, the two shares, and the recovered image.

Fig. 11.2 shows the pipeline on a document-like test image. Neither share reveals any structure; the recovery is visually identical to the input. Part 1 of this project recorded as a limitation that reconstructed images showed "measurable PSNR and SSIM degradation". That statement is incorrect, and the correction matters because it changes what the enhancement objective was supposed to fix.

Table 11.2: Reconstruction fidelity over six test images, covering synthetic patterns and real application assets. Every reconstruction is bit-exact.

Test image	Size	Bit-exact	MSE	PSNR (dB)	SSIM	Differing pixels
gradient	256x256	Yes	0.0	infinite	1.0000	0
checkerboard	256x256	Yes	0.0	infinite	1.0000	0
document	256x256	Yes	0.0	infinite	1.0000	0
natural-like	256x256	Yes	0.0	infinite	1.0000	0
logo	256x256	Yes	0.0	infinite	1.0000	0
app-icon	256x256	Yes	0.0	infinite	1.0000	0

Table 11.2 reports the result over six images: four synthetic patterns chosen to stress different statistics (a linear gradient, a checkerboard, a text document and a smooth natural-like field) and two real assets taken from the application itself. In every case the recovered array is identical to the input, pixel for pixel. Mean squared error is exactly zero, so peak signal-to-noise ratio is unbounded and the structural similarity index [25] is exactly 1.0000.

This is a property of the scheme, not a favourable measurement. XOR is its own inverse, so $(O \text{ XOR } S1) \text{ XOR } S1 = O$ for every byte, with no rounding step anywhere in the path. Lossy behaviour can only enter through a lossy transport or storage format, which is why shares are handled as PNG and never as JPEG.

11.3.3 Statistical security of the shares

Bit-exact recovery says the scheme is correct. It says nothing about whether a single share leaks the secret. Three independent statistical tests were run to check that it does not.

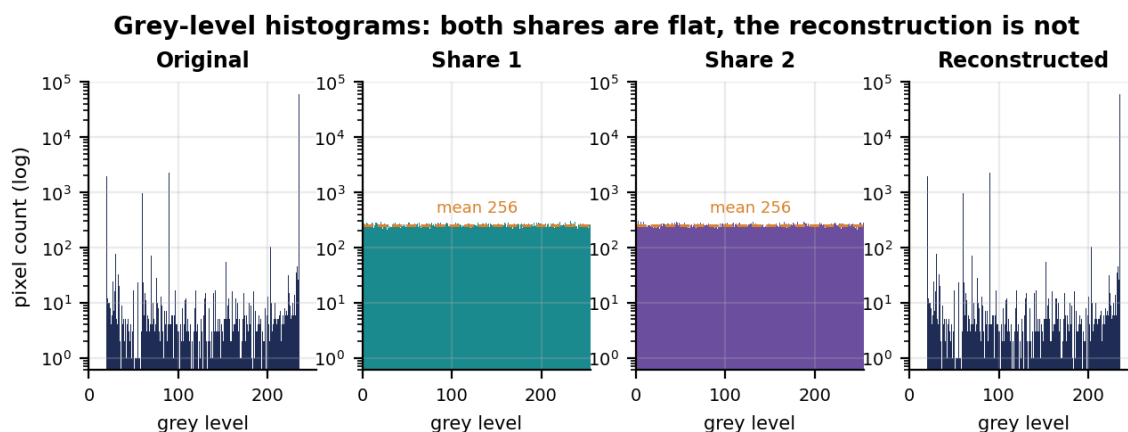


Figure 11.3: Grey-level histograms on a logarithmic count axis. The original and the recovery are strongly peaked; both shares are flat at the uniform expectation.

Adjacent-pixel correlation is destroyed by the split
(4000 sampled horizontal pairs, "natural-like" test image)
Original image **Share 1**
horizontal correlation $r = 0.9788$ **horizontal correlation $r = 0.0035$**

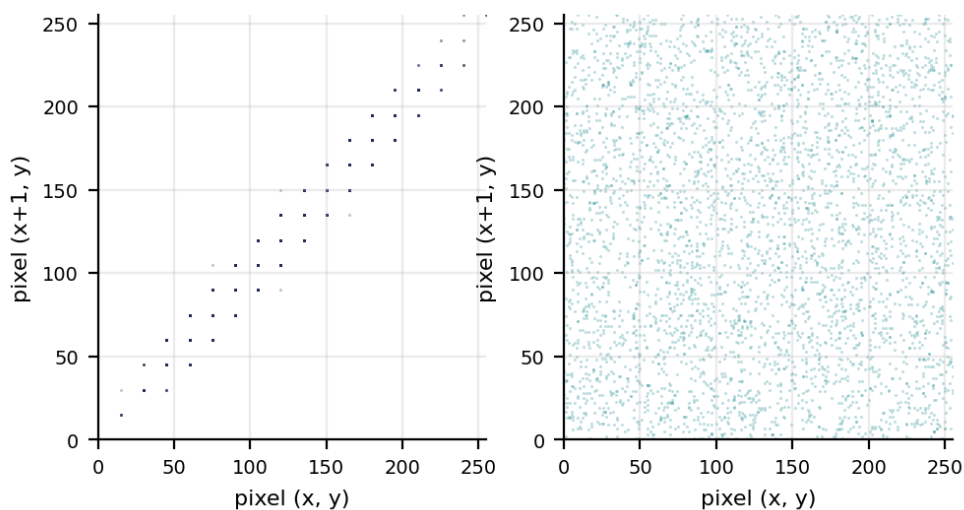


Figure 11.4: Adjacent horizontal pixel pairs, 4000 samples. Natural images lie on the diagonal; a share fills the square uniformly.

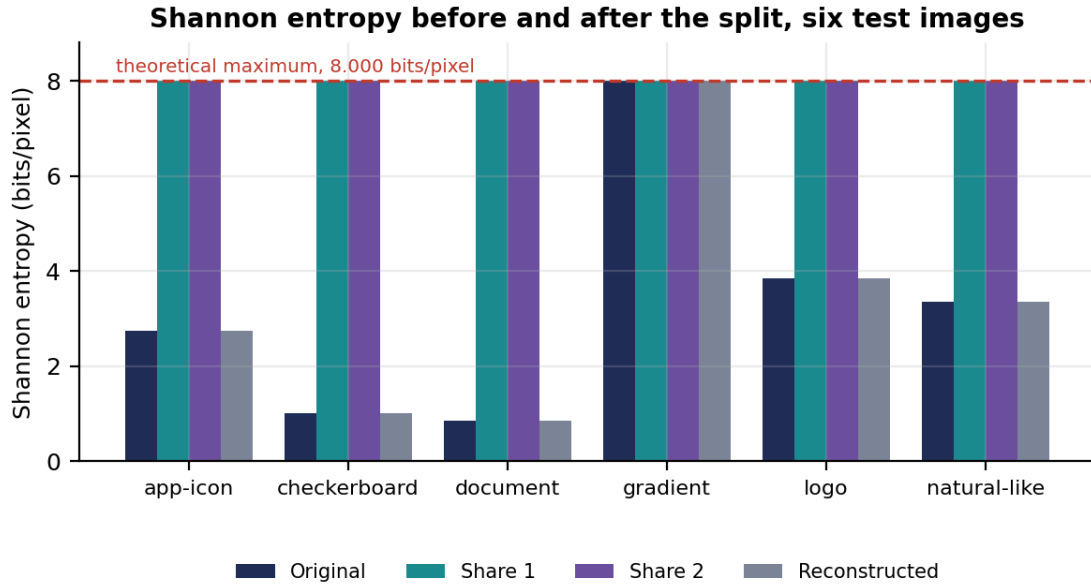


Figure 11.5: Shannon entropy of each array across the six test images. Both shares sit at the 8.000 bits/pixel maximum regardless of the content they conceal.

The first test is the grey-level histogram (Fig. 11.3). A natural or document image concentrates its mass on a few levels; both shares are flat across all 256 levels at the uniform expectation. A chi-square goodness-of-fit test against the uniform distribution fails to reject uniformity for every one of the 12 share arrays measured, with p values from 0.117 to 0.764, and rejects it decisively for every original except one.

That exception is worth a sentence, because it is a useful warning against reading too much into a single test. The gradient test image is a linear ramp, so it visits every grey level the same number of times and its histogram is uniform by construction - the chi-square test cannot distinguish it from random noise at all. Its adjacent-pixel correlation, however, is almost 1. A flat histogram is necessary for a share to look random, but it is nowhere near sufficient, which is exactly why the detector in Section 11.4 measures correlation and entropy alongside histogram shape rather than relying on any one of them.

The second is adjacent-pixel correlation (Fig. 11.4), the standard check in the image-encryption literature. On the natural-like test image the horizontal correlation of the original is $r = 0.9788$, the strong diagonal expected of real image content. On its Share 1 the same statistic is $r = 0.0035$, which is statistically indistinguishable from zero at this sample size. The structure a human or an algorithm would exploit has been removed, not merely hidden.

The third is Shannon entropy [8] (Fig. 11.5). A uniformly distributed 8-bit image has an entropy of exactly 8 bits per pixel, and no image can exceed it. Both shares reach that bound to four decimal places on every test image, including the checkerboard whose original entropy is near 1 bit per pixel. The measured entropy of a share is independent of what it conceals, which is precisely the property required.

Table 11.3: Statistical profile of the originals, the shares and the recovery for two representative test images. The share rows are indistinguishable from uniform random data; the original and recovery rows are identical to each other.

Test image	Array	Entropy (bits/px)	Mean	Corr. h	Corr. v	Chi-square p	Uniform?
document	original	0.8494	218.5	0.8962	0.8737	0.0000	no
document	share 1	7.9972	127.5	-0.0037	-0.0092	0.4580	yes

Test image	Array	Entropy (bits/px)	Mean	Corr. h	Corr. v	Chi-square p	Uniform?
document	share 2	7.9972	127.4	-0.0017	-0.0122	0.5696	yes
document	reconstructed	0.8494	218.5	0.8962	0.8737	0.0000	no
natural-like	original	3.3539	128.4	0.9788	0.9796	0.0000	no
natural-like	share 1	7.9973	127.8	-0.0052	-0.0005	0.6440	yes
natural-like	share 2	7.9973	127.2	-0.0043	-0.0033	0.6486	yes
natural-like	reconstructed	3.3539	128.4	0.9788	0.9796	0.0000	no

Table 11.3 collects the three tests for two representative images. The pattern repeats for all six. Note that the "original" and "reconstructed" rows of each image are identical in every column, which is the same bit-exactness result arriving from a different direction.

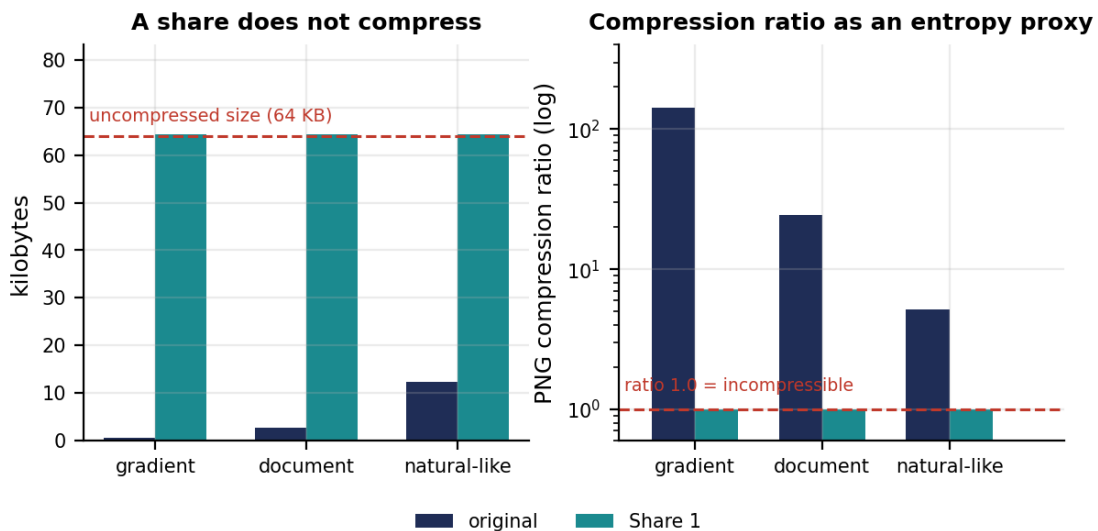


Figure 11.6: Shares do not compress. PNG output size and compression ratio for three originals and their Share 1.

Table 11.4: Storage cost of a share. A share is incompressible, so storing it costs the full uncompressed size of the image regardless of how compressible the original was.

Test image	Raw bytes	PNG original	PNG Share 1	Original ratio	Share ratio	Storage overhead
gradient	65536	461	65976	142.2x	0.993x	143x
document	65536	2672	65976	24.5x	0.993x	25x
natural-like	65536	12668	65976	5.2x	0.993x	5x

A fourth piece of evidence falls out of the implementation rather than from a deliberate test (Fig. 11.6, Table 11.4). PNG uses lossless entropy coding, so its output size is a practical upper bound on the compressibility of an array. The gradient image compresses 142-fold; its share compresses by a factor of 0.993, meaning the encoder cannot find any redundancy at all and the file is slightly larger than the raw pixels because of PNG framing overhead. An incompressible file is an empirical statement about entropy that does not depend on any statistical model.

This has an engineering consequence that should be stated: storing Share 1 costs the full uncompressed size of the image. For the gradient test image that is a 143-fold storage increase

over the compressed original. The one-hour message time-to-live in the application exists partly for this reason.

11.3.4 A limitation of the scheme: no diffusion

One property of XOR visual cryptography is worth recording as a limitation because it is easy to overstate the scheme's strength. A modern block cipher exhibits the avalanche effect: changing one bit of plaintext changes roughly half the ciphertext bits, and the standard measure of this (NPCR) is expected to be about 99.609%. The XOR split has no such property.

Measured directly: altering a single pixel of the original and re-splitting with the same Share 1 changes exactly 1 pixel of Share 2, an NPCR of 0.001526%. This is not a defect, because confidentiality rests on the pad being uniform and used once, not on diffusion. But it does mean that an attacker who holds Share 2 and can observe how it changes across retransmissions of a near-identical image learns the positions that changed. Pad reuse would be catastrophic for the same reason it is catastrophic for any one-time pad [9], which is why Share 1 is drawn afresh for every message and never cached.

11.4 Machine-learned tamper detection: design

11.4.1 Why a classifier at all

A keyed message authentication code already detects any modification to a stored share, and Section 11.9 shows it doing so without a single miss. The obvious question - why add a classifier - has a specific answer. An HMAC [11] is a binary oracle over the whole file: it reports that something changed, never what or where, and it is useless once a share has been legitimately re-encoded. A statistical classifier is approximate, but it localises damage, produces a calibrated confidence, and survives transformations that preserve content. Section 11.9 shows the two failing in different places, which is the empirical argument for keeping both.

11.4.2 Feature set

The detector does not operate on raw pixels. It extracts 13 statistical features, chosen because a valid Share 1 has a known expected profile: uniform histogram, zero adjacent correlation, maximum entropy, mean near 127.5. Every feature measures a departure from that profile. The approach follows the rich-model tradition in image forensics [27], at a much smaller scale appropriate to a feature vector that must also run in a browser.

Table 11.5: The 13-feature vector, its interpretation, and the tampering signal each feature is intended to capture.

#	Feature	What it measures	Why a tampered share moves it
1	mean	Average grey level	Brightness shifts and block fills pull the mean away from 127.5
2	std	Standard deviation of grey levels	Flat regions reduce spread; clipping increases it
3	entropy	Shannon entropy over the 256-bin histogram	Any inserted structure costs entropy; a clean share sits at 8.000
4	hist_chi2	Chi-square distance of the histogram from uniform	Edits concentrate mass on particular grey levels
5	hist_l1_uniform	L1 distance of the histogram from uniform	A robust companion to the chi-square term, less dominated by outliers
6	corr_h	Correlation of horizontally adjacent pixels	Blur and resampling introduce neighbour correlation where there was none
7	corr_v	Correlation of vertically adjacent pixels	As above, in the orthogonal direction

#	Feature	What it measures	Why a tampered share moves it
8	laplacian_var	Variance of the Laplacian response	A smoothness proxy: blurring lowers it, scratches raise it locally
9	block_mean_std	Spread of 16x16 block means	A uniform share has near-identical block means; a patch breaks that
10	block_std_mean	Average of 16x16 block standard deviations	Detects locally flattened regions that global statistics hide
11	unique_ratio	Distinct grey levels present, over 256	Block replacement and quantisation reduce the palette
12	zero_ratio	Fraction of pure black pixels	Block fills and clipping create runs of exact zeros
13	sat_ratio	Fraction of pure white pixels	The saturated counterpart of the previous feature

Table 11.5 lists the features. Three design constraints shaped the set. It must be cheap enough to run on every share before reconstruction; it must be reproducible bit-for-bit in JavaScript so the same model can run in the browser (Section 11.13); and it must be interpretable, so the application can tell a user why a share was flagged rather than only that it was.

11.4.3 Attack model

Training and evaluation both use synthesised tampering. The seven attacks were chosen to span the plausible ways a stored share is damaged or manipulated, from deliberate editing to incidental format loss.

Table 11.6: The seven simulated attacks used for both training and evaluation, with the real-world case each stands in for.

Attack	Operation	Represents
block	Replace a rectangular region with a constant grey level	Deliberate censorship or region substitution
noise	Replace a fraction of pixels with fresh random values	Bit-level corruption, or an attacker mimicking share statistics
blur	Gaussian blur applied to a sub-region	Smoothing to erase detail; resampling artefacts
brightness	Add a constant offset to a sub-region, then clip	Exposure or level manipulation
crop	Crop the border away and rescale to the original size	Reframing, and the resampling it forces
scratch	Draw several thick random lines	Overwriting or defacing part of the share
jpeg	Encode and decode as JPEG at a reduced quality factor	Lossy re-encoding by a well-meaning intermediary

Table 11.6 defines them. Attack strength is sampled uniformly from 0.18 to 0.55 so the classifier sees a range of severities rather than one convenient setting. The noise attack is examined separately and in much more detail in Section 11.8, because it is where the classifier fails.

11.4.4 Datasets and evaluation protocol

Three datasets are used, and keeping them separate is what makes the results in this chapter meaningful. Reporting a model's accuracy on data it was fitted to is the most common way an evaluation becomes worthless.

Table 11.7: The three datasets. *D_eval* is 14 times the size of the internal test split and was never seen during fitting.

Dataset	Seed	Samples	Composition	Used for
D_train	42	240	120 clean + 120 tampered, paired	Fitting the shipped model (75%% split) and all comparison models
D_eval	2026	840	420 clean + 420 tampered (60 per attack)	Every accuracy, ROC and per-attack number in this chapter
D_noise	7331	400	40 samples at each of 10 noise strengths	The blind-spot analysis in Section 11.8

Table 11.7 describes them. Two details of the construction matter. First, every tampered sample is derived from the clean sample that sits next to it in the dataset, so the only difference between a negative and a positive example is the attack itself - share-to-share variation is removed from the comparison by construction. Second, each share is 96x96 pixels, which is the scale at which the model was trained to operate; feature values are ratios and correlations rather than absolute counts, so they transfer to other sizes.

The shipped model is a RandomForest [21] of 45 trees with a maximum depth of 10, class-weighted to balance the two classes, holding 1109 decision nodes in total at a mean realised depth of 7.7. It is 107.7 KB as a joblib file on the server and 40.2 KB once exported to ONNX for the browser. The implementation is scikit-learn [22] over NumPy [23] and OpenCV [24].

11.5 Detection results

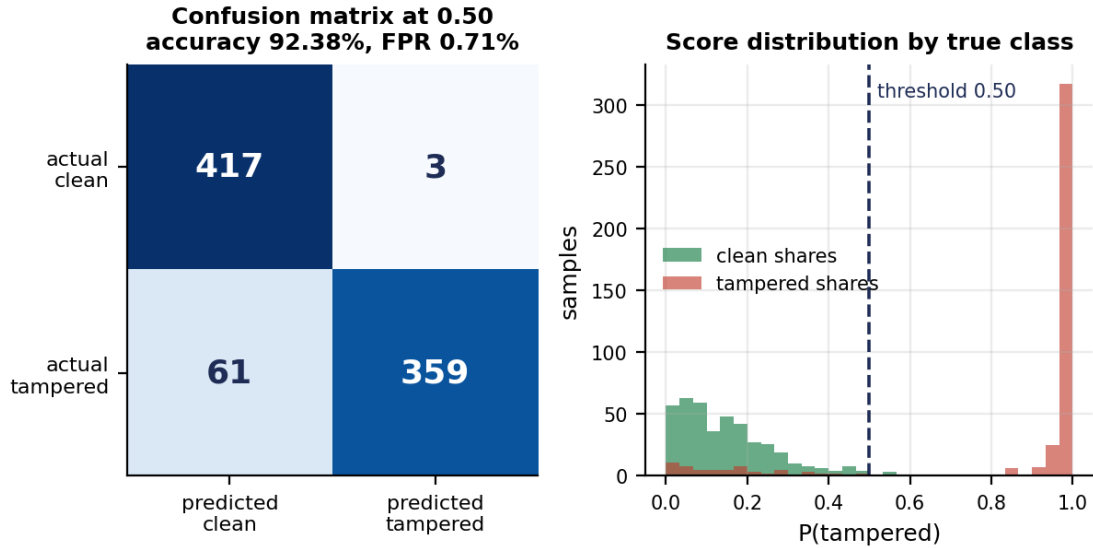


Figure 11.7: Confusion matrix at the shipped 0.50 threshold, and the distribution of predicted probabilities by true class.

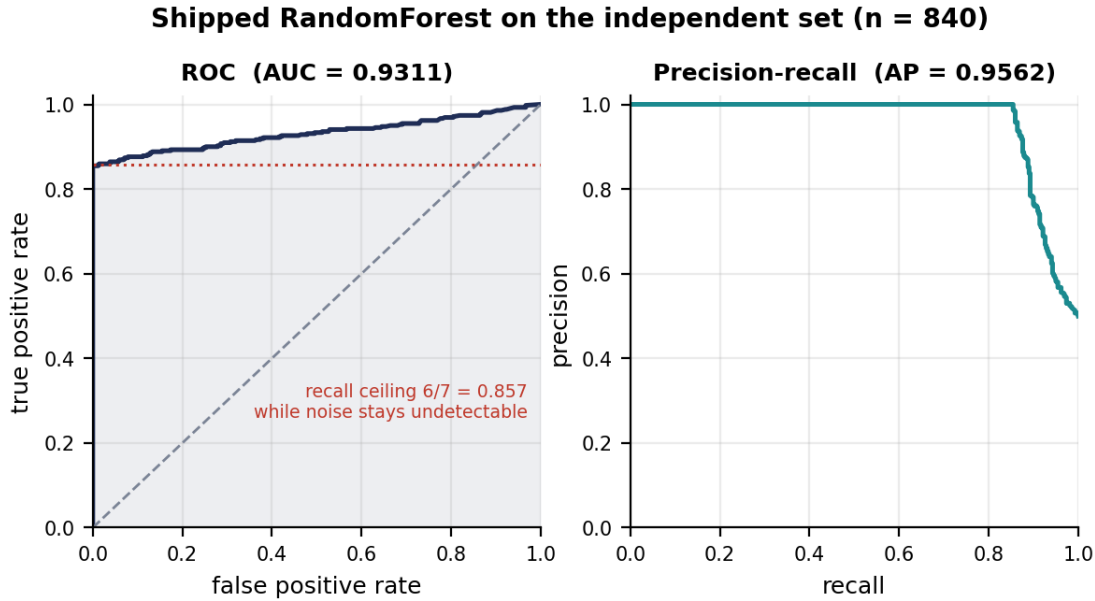


Figure 11.8: ROC and precision-recall curves on the independent set. The dotted line marks the recall ceiling imposed by the noise blind spot.

Table 11.8: Headline detection performance on the independent set (n = 840), at three decision thresholds.

Metric	Threshold 0.50 (shipped)	Threshold 0.35	Threshold 0.60
Accuracy	92.38%	90.00%	92.74%
Precision	99.17%	92.42%	100.00%
Recall	85.48%	87.14%	85.48%
F1 score	0.9182	0.8971	0.9217
False positive rate	0.71%	7.14%	0.00%
True positives / False negatives	359 / 61	366 / 54	359 / 61
True negatives / False positives	417 / 3	390 / 30	420 / 0

On the independent set the shipped model reaches an accuracy of 92.38% at its default threshold, with a precision of 99.17% and a recall of 85.48% (Fig. 11.7, Table 11.8). The area under the ROC curve is 0.9311 and average precision is 0.9562 [26]. Of 420 clean shares, 3 were misclassified: a false positive rate of 0.71%.

The score distribution in Fig. 11.7 is more informative than the accuracy figure. The two classes are almost perfectly separated: tampered shares pile up at a probability of 1.0 and clean shares at 0.0, with very little mass in between. The 61 errors are not borderline cases that a better threshold would fix - they are a specific, identifiable subset, and Section 11.8 identifies it.

Fig. 11.8 shows the ROC curve rising almost vertically to a true positive rate of about 0.85 and then flattening. That plateau is not a property of the classifier's calibration; it is the recall ceiling imposed by one attack the features cannot see, and no threshold anywhere on the curve lifts it.

11.6 Detection by attack type

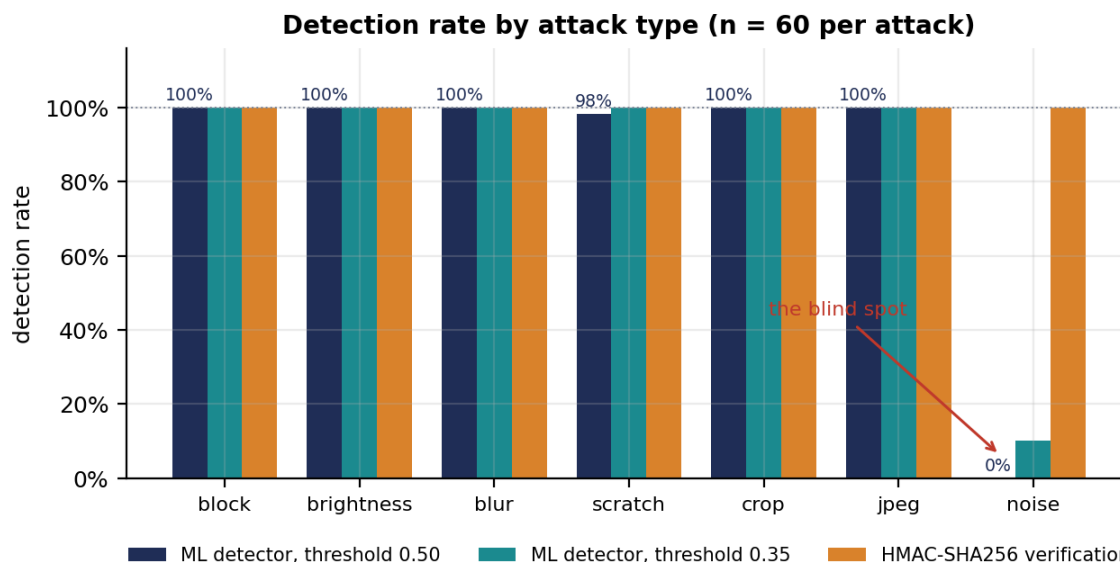


Figure 11.9: Detection rate by attack type at two thresholds, against the HMAC verification rate on the same attacks.

Table 11.9: Per-attack detection on the independent set. Six attacks are detected essentially without error; one is not detected at all.

Attack	n	Mean P(tampered)	Detected at 0.50	Rate at 0.50	Detected at 0.35	Rate at 0.35
block	60	1.000	60 / 60	100.0%	60 / 60	100.0%
brightness	60	0.995	60 / 60	100.0%	60 / 60	100.0%
blur	60	0.987	60 / 60	100.0%	60 / 60	100.0%
scratch	60	0.950	59 / 60	98.3%	60 / 60	100.0%
crop	60	1.000	60 / 60	100.0%	60 / 60	100.0%
jpeg	60	0.968	60 / 60	100.0%	60 / 60	100.0%
noise	60	0.158	0 / 60	0.0%	6 / 60	10.0%
clean (false positives)	420	0.149	3 / 420	0.7%	30 / 420	7.1%

Aggregate accuracy hides the structure of the failures, so Fig. 11.9 and Table 11.9 break the result down by attack. Six of the seven attacks are detected at or very near 100%: block replacement, brightness shift, blur, crop-and-rescale and JPEG re-encoding are each caught in every one of 60 trials, and scratching is caught in 59 of 60. The mean predicted probability for these six ranges from 0.950 to 1.000.

The seventh is the random-noise attack, detected in 0 of 60 trials at the shipped threshold. It accounts for 60 of the 61 false negatives in the confusion matrix; the remaining 1 is the single missed scratch. In other words, almost the entire error budget of this classifier is one attack type.

That observation yields a useful analytical check. If the noise attack were undetectable in principle and everything else were classified perfectly with no false positives, accuracy on this dataset would be 0.9286. The measured accuracy at the shipped threshold is 0.9238, and at a threshold of 0.60 it is 0.9274 - within 0.1 percentage points of that ceiling. The classifier is therefore not under-trained: it is performing at close to the maximum its feature set permits, and the remaining gap is a representation problem rather than a learning problem.

11.7 Which features carry the signal

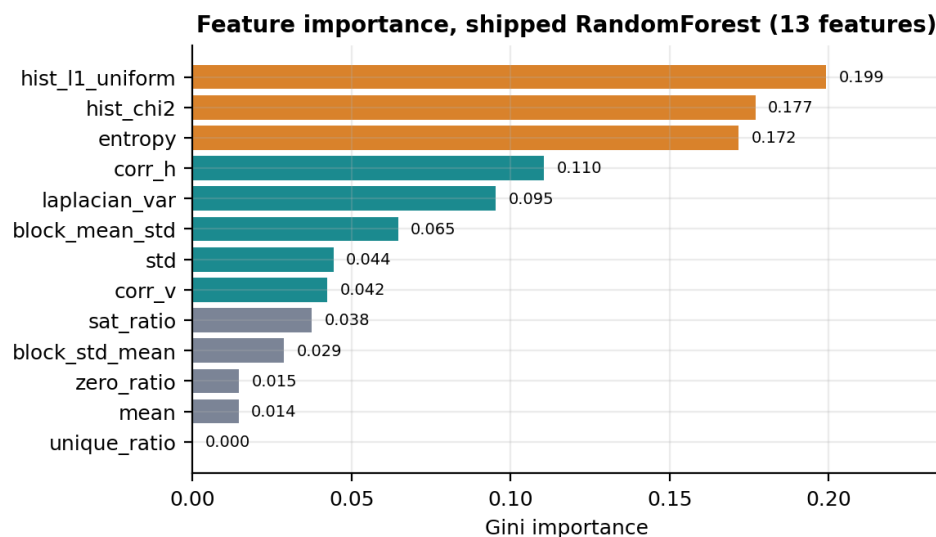


Figure 11.10: Gini importance of the 13 features in the shipped RandomForest.

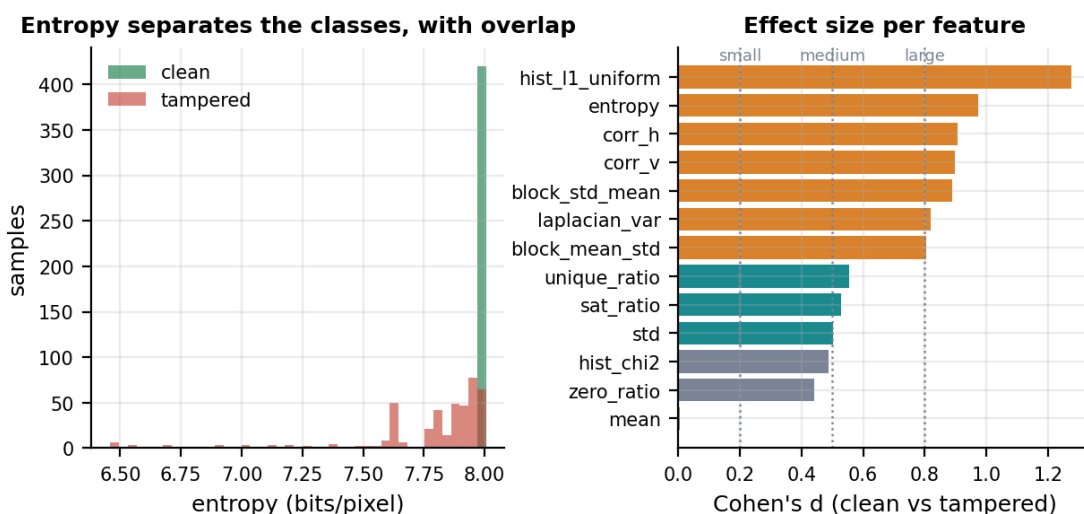


Figure 11.11: Left: the entropy distribution of clean and tampered shares. Right: effect size per feature, measured as Cohen's d.

Table 11.10: The eight most important features, with the effect size and significance of the clean-versus-tampered difference measured independently on D_{eval} .

Rank	Feature	Gini importance	Clean mean	Tampered mean	Cohen's d	Mann-Whitney p
1	hist_l1_uniform	0.1993	0.1321	0.2749	1.277	4.0e-101
2	hist_chi2	0.1770	0.9915	49.3952	0.487	6.1e-104
3	entropy	0.1717	7.9800	7.7800	0.974	1.2e-103
4	corr_h	0.1104	-0.0012	0.1139	0.907	1.7e-31
5	laplacian_var	0.0953	109771.6264	91210.6069	0.819	9.5e-20
6	block_mean_std	0.0646	4.4816	10.6727	0.804	6.0e-37
7	std	0.0443	73.8858	70.5187	0.503	1.2e-08
8	corr_v	0.0423	-0.0002	0.1147	0.899	6.9e-31

Fig. 11.10, Fig. 11.11 and Table 11.10 examine what the model is actually using. The top three features - hist_l1_uniform, hist_chi2 and entropy - account for 54.8% of total Gini importance between them, and all three measure the same underlying thing from different directions: how far the histogram has moved away from uniform.

Importance alone can be misleading, because a RandomForest will distribute credit arbitrarily among correlated features. The effect sizes in Table 11.10 are computed independently of the model, directly on the evaluation set. They agree with the importance ranking: hist_l1_uniform separates the classes with a Cohen's d of 1.28 and entropy with 0.97, both far above the 0.8 threshold conventionally called a large effect, and a Mann-Whitney U test rejects the null hypothesis of equal distributions at $p < 1e-30$ for all of the leading features.

The left panel of Fig. 11.11 also explains the blind spot visually. Clean shares form a tight spike at the 8.000-bit maximum. Tampered shares spread to the left of it. An attack that leaves entropy at the maximum therefore produces a feature vector that lands inside the clean cluster - and replacing uniform random pixels with different uniform random pixels does exactly that.

11.8 The noise blind spot, mapped

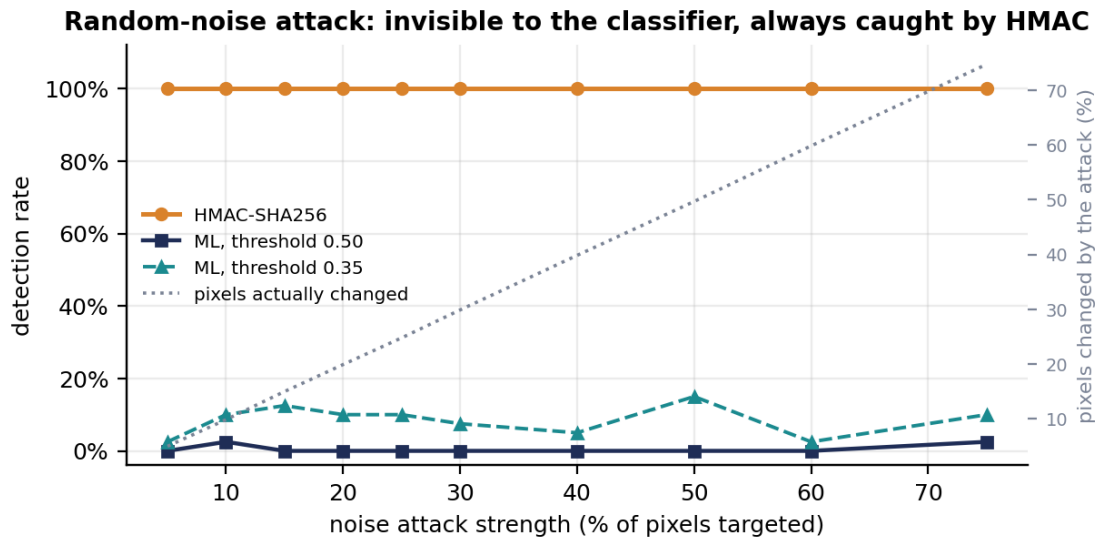


Figure 11.12: Noise attack across ten strengths: detection by the classifier and by HMAC, with the fraction of pixels the attack actually changed.

Table 11.11: The noise attack swept from 5%% to 75%% strength, 40 trials per level. The attack is real at every level and invisible to the classifier at every level.

Strength	Pixels changed	Mean abs. delta	Mean P(tampered)	Max P(tampered)	ML rate at 0.50	ML rate at 0.35	HMAC rate
5%	5.0%	4.3	0.145	0.429	0.0%	2.5%	100.0%
10%	9.9%	8.5	0.156	0.507	2.5%	10.0%	100.0%
15%	15.0%	12.8	0.176	0.469	0.0%	12.5%	100.0%
20%	19.9%	17.0	0.137	0.463	0.0%	10.0%	100.0%
25%	24.8%	21.3	0.157	0.452	0.0%	10.0%	100.0%
30%	29.9%	25.6	0.156	0.384	0.0%	7.5%	100.0%
40%	39.9%	34.1	0.167	0.429	0.0%	5.0%	100.0%
50%	49.7%	42.4	0.169	0.476	0.0%	15.0%	100.0%

Strength	Pixels changed	Mean abs. delta	Mean P(tampered)	Max P(tampered)	ML rate at 0.50	ML rate at 0.35	HMAC rate
60%	59.8%	51.4	0.152	0.374	0.0%	2.5%	100.0%
75%	74.7%	64.0	0.168	0.526	2.5%	10.0%	100.0%

A single failing attack deserves to be understood rather than reported. The first hypothesis was that the training strength range simply did not cover the cases being tested, which a wider sweep would fix. Fig. 11.12 and Table 11.11 test that directly, across strengths from 5% to 75% at 40 trials each.

The hypothesis is wrong. Detection never exceeds 2.5% at the shipped threshold anywhere on the sweep, and the mean predicted probability stays between 0.137 and 0.176 throughout - essentially flat, and essentially identical to the 0.149 mean the model assigns to genuinely clean shares. Strength does not help, because strength is not the variable that matters.

The attack is unquestionably happening. The third column of Table 11.11 confirms that the measured fraction of changed pixels tracks the requested strength almost exactly, up to 74.7% of all pixels at the strongest setting, with a mean absolute change of 64.0 grey levels. Three quarters of the image is being overwritten and the classifier still sees nothing.

The explanation is structural, and it generalises beyond this implementation. The feature vector measures departures from uniform randomness. The noise attack replaces uniform random pixels with fresh uniform random pixels. The result is still uniform random, so every one of the 13 features is unchanged in distribution and the feature vector is, by construction, that of a clean share. No amount of additional training data and no alternative classifier over these features can fix this, because the two classes are genuinely identical in this representation. Section 11.11 confirms that prediction against seven other classifiers.

This is the single most useful result in the Part 2 evaluation, and the reason is worth stating plainly: a system with only a machine-learned integrity check would be fully exploitable by an attacker who knew this, and the exploit is trivial. The next section shows why Reveal-X is not.

11.9 Two-layer integrity: HMAC and ML together

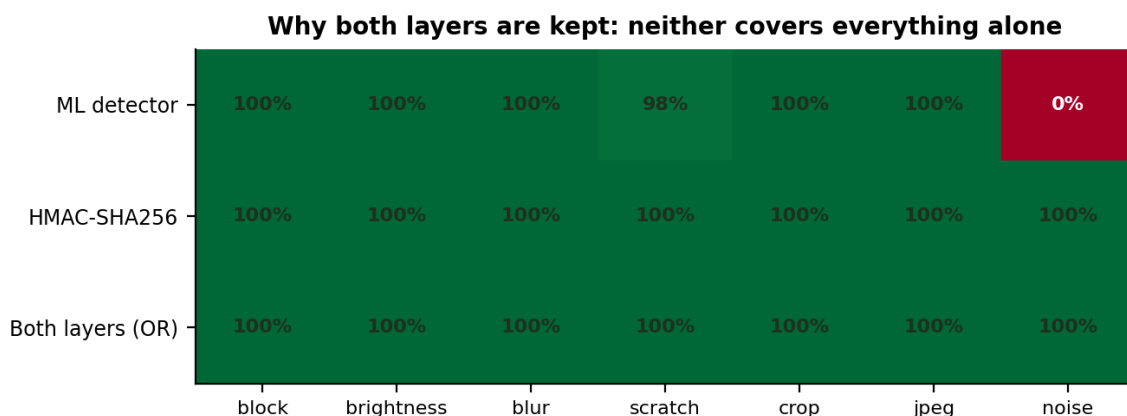


Figure 11.13: Coverage matrix. Neither layer detects every attack; the union of the two does.

Table 11.12: The two integrity layers compared on identical attacks. HMAC catches everything including the classifier's blind spot; the classifier adds localisation and survives re-encoding.

Attack	ML detected	ML rate	HMAC detected	HMAC rate	Either layer
block	60 / 60	100.0%	60 / 60	100.0%	100.0%
brightness	60 / 60	100.0%	60 / 60	100.0%	100.0%
blur	60 / 60	100.0%	60 / 60	100.0%	100.0%
scratch	59 / 60	98.3%	60 / 60	100.0%	100.0%
crop	60 / 60	100.0%	60 / 60	100.0%	100.0%
jpeg	60 / 60	100.0%	60 / 60	100.0%	100.0%
noise	0 / 60	0.0%	60 / 60	100.0%	100.0%
clean shares accepted	-	-	120 / 120	100.0%	-

Each share is stored with an HMAC-SHA256 tag [11] computed over the bytes as written to disk, and compared on retrieval with a constant-time comparison. Fig. 11.13 and Table 11.12 run both layers over the same attacks.

HMAC detects 420 of 420 tampered shares across all seven attacks, including 60 of 60 noise attacks, and accepts 120 of 120 untouched shares. This is the expected behaviour of a secure MAC rather than a surprising result: any change to any byte changes the tag, and forging a valid tag without the key is the security property the construction is built on.

So the layers fail in different places, and that is the entire argument for carrying both. HMAC is exact but says only that something changed - no location, no classification, and no answer at all once a share has been legitimately re-encoded, since a JPEG round trip breaks the tag just as thoroughly as an attack does. The classifier survives re-encoding, localises the damage to a region, and explains its verdict in terms a user can read, but it is blind to the one attack HMAC handles most easily. Neither is redundant.

Case study: one block attack, detected, localised, and visible in the output

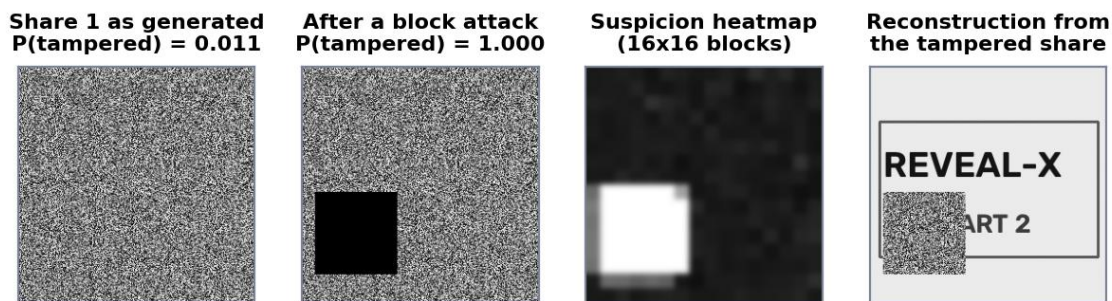


Figure 11.14: One block attack end to end: the clean share, the tampered share, the suspicion heatmap, and the visibly corrupted reconstruction.

Fig. 11.14 follows a single block attack through the pipeline. The clean share is scored at $P(\text{tampered}) = 0.011$. After the attack the same share scores 1.0000, the 16x16 block heatmap localises the damage to the correct region, and the HMAC check fails. The reconstruction from the tampered share shows the damage directly, which is what the user would otherwise have seen without explanation.

The detector also returns the reasons behind its verdict. For this sample they were:

- Entropy is lower than a valid random VC share, suggesting inserted structure or edited pixels.
- Pixel mean is shifted away from the expected random-share centre.
- Neighbouring pixels show unusual correlation for a random share.
- Large numbers of pure black/white pixels indicate block replacement or clipping.

11.10 Operating point selection

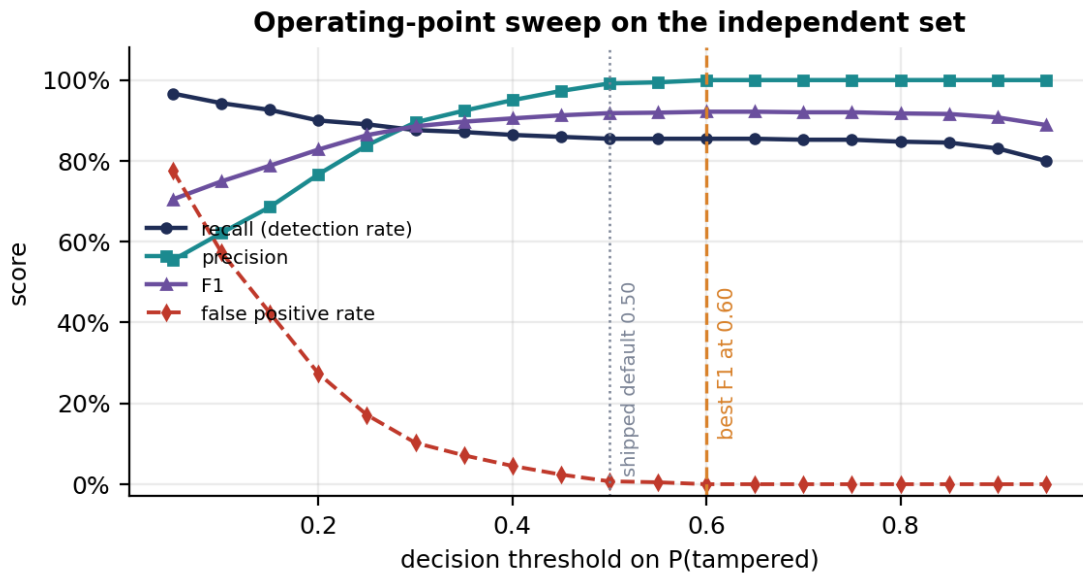


Figure 11.15: Recall, precision, F1 and false positive rate against the decision threshold.

Table 11.13: Operating points. Raising the threshold from 0.50 to 0.60 removes every false positive without costing a single detection.

Threshold	Accuracy	Precision	Recall	F1	FPR	TP	FP	FN	TN
0.10	68.45%	62.17%	94.29%	0.7493	57.38%	396	241	24	179
0.20	81.31%	76.67%	90.00%	0.8280	27.38%	378	115	42	305
0.30	88.69%	89.54%	87.62%	0.8857	10.24%	368	43	52	377
0.35	90.00%	92.42%	87.14%	0.8971	7.14%	366	30	54	390
0.40	90.95%	95.03%	86.43%	0.9052	4.52%	363	19	57	401
0.50	92.38%	99.17%	85.48%	0.9182	0.71%	359	3	61	417
0.60	92.74%	100.00%	85.48%	0.9217	0.00%	359	0	61	420
0.70	92.62%	100.00%	85.24%	0.9203	0.00%	358	0	62	420
0.80	92.38%	100.00%	84.76%	0.9175	0.00%	356	0	64	420
0.90	91.55%	100.00%	83.10%	0.9077	0.00%	349	0	71	420

The shipped threshold of 0.50 is the scikit-learn default rather than a choice the project made, so it was worth testing. Fig. 11.15 and Table 11.13 sweep it.

The result is an unambiguous recommendation. At 0.60 the model achieves 100.0% precision with 85.48% recall - identical recall to the 0.50 threshold, but with the 3 false positives eliminated entirely. Accuracy rises from 92.38% to 92.74%, which is 0.0012 short of the analytical ceiling derived in Section 11.6. The separation between the classes is wide enough that the exact

threshold barely matters between about 0.4 and 0.8, which is itself evidence of a well-separated problem.

Lowering the threshold to 0.35 is the opposite trade: recall improves by only 1.7 percentage points, because the samples it recovers are noise attacks that sit nowhere near the boundary, while false positives rise from 3 to 30. For an integrity warning shown to a user, that is the wrong direction - a false alarm on a clean share costs trust, and the recall it buys is negligible.

11.11 Model selection: was RandomForest the right choice?

Table 11.14: Eight classifiers trained on identical features and the same split, evaluated on the same independent set, with the rule-based fallback for reference.

Classifier	5-fold CV accuracy	Internal test	D_eval accuracy	Precision	Recall	F1	ROC AUC	Fit (ms)
k-NN (k=5)	0.921 +/- 0.036	0.933	0.9286	0.997	0.860	0.923	0.9335	3.7
ExtraTrees	0.929 +/- 0.031	0.950	0.9274	1.000	0.855	0.922	0.9315	27.3
SVM (RBF)	0.917 +/- 0.037	0.950	0.9274	1.000	0.855	0.922	0.9378	15.1
MLP (64,32)	0.929 +/- 0.031	0.950	0.9274	1.000	0.855	0.922	0.9366	86.0
LogisticRegression	0.929 +/- 0.031	0.950	0.9262	1.000	0.852	0.920	0.9378	8.7
RandomForest (shipped)	0.925 +/- 0.036	0.933	0.9238	0.992	0.855	0.918	0.9311	45.2
GradientBoosting	0.921 +/- 0.024	0.950	0.9024	0.936	0.864	0.899	0.9257	62.9
DecisionTree	0.867 +/- 0.021	0.900	0.8667	0.865	0.869	0.867	0.8871	1.1
Statistical fallback (no model file)	-	-	0.6786	1.000	0.357	0.526	0.9048	-

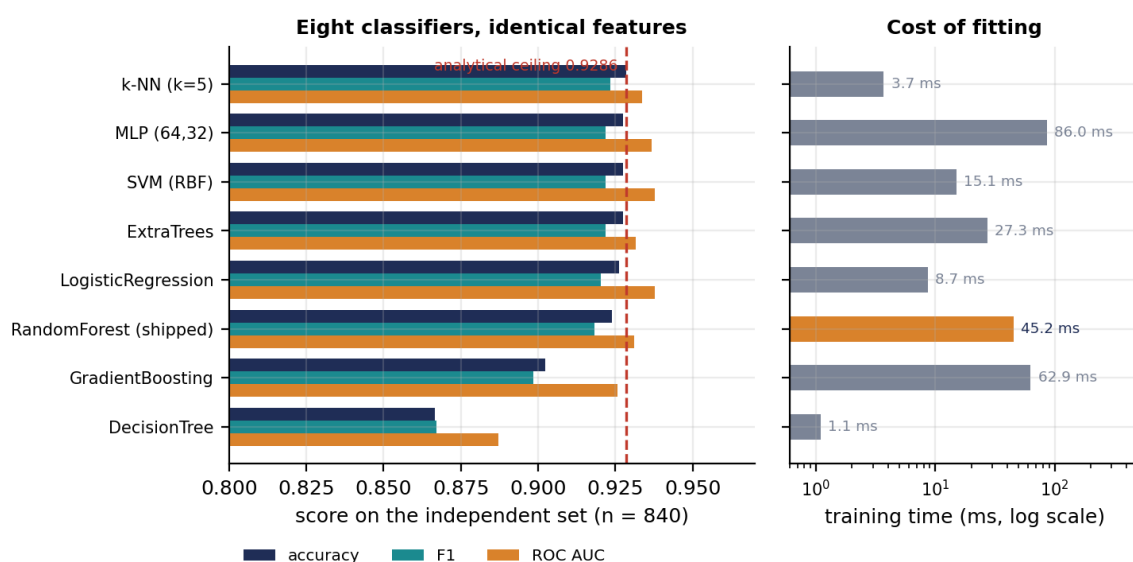


Figure 11.16: Eight classifiers on identical features. All cluster at the ceiling imposed by the blind spot.

A reviewer is entitled to ask whether a different model would do better. Table 11.14 and Fig. 11.16 answer it: eight classifiers spanning linear, kernel, instance-based, tree and neural families, all trained on the same features and the same split, all evaluated on the same independent set.

The spread is narrow. Seven of the eight land between 0.9024 and 0.9286 accuracy, and the best (k-NN (k=5) at 0.9286) beats the shipped RandomForest by 0.48 percentage points - a difference of 4 samples out of 840, which is well inside sampling noise. Only the single decision tree is clearly worse at 0.8667, as expected from an unensembled learner on 13 correlated features.

That clustering is the real finding, and it confirms the prediction made in Section 11.8. Every model converges on the same ceiling because every model is reading the same 13 numbers, and in that representation the noise attack is simply not present. The limitation lives in the features, not in the learner - so changing the learner is not where effort should go.

The RandomForest is nevertheless the right thing to ship, for reasons the accuracy column does not show. It exports cleanly to ONNX for in-browser execution, it yields the per-feature importances used in Section 11.7, and it produces the human-readable explanations the interface displays. The MLP and SVM match it on accuracy while offering none of those properties.

The last row of Table 11.14 is the rule-based fallback that runs when the model file is absent. At 67.9% accuracy and 35.7% recall it is substantially worse than the trained model, which quantifies what the machine-learning component is actually contributing: 24.5 percentage points of accuracy and 49.8 points of recall over a hand-tuned heuristic on the same features. Notably its precision is 100% - it never produces a false alarm - which is the correct conservative behaviour for a degraded mode.

11.12 Objective 9: not delivered, and why

Objective 9 specified a denoising convolutional neural network and ESRGAN super-resolution for the reconstructed image. Neither was implemented. The enhancement stage in the codebase remains a classical pipeline - bilateral filtering, unsharp masking and CLAHE - and this section states the reason rather than leaving the omission unexplained.

The objective was written on the assumption recorded in Part 1, following the enhancement approach surveyed there [40]: that reconstruction was lossy and the recovered image needed restoring. Section 11.3 shows that assumption to be false. Reconstruction is bit-exact, with MSE of exactly zero and SSIM of exactly 1.0000 on every test image. A denoising network such as DnCNN [30], trained by the standard supervised procedure [38], learns a mapping from a degraded image to a clean one; here the input and the target are the same array, so the optimal learned mapping is the identity and the training objective is degenerate. There is nothing for the network to remove.

The same reasoning applies with less force to the super-resolution half. ESRGAN [32], building on SRGAN [31], upscales a low-resolution image plausibly rather than faithfully - it synthesises detail that was not present. For a security application whose purpose is to establish whether an image has been altered, adding invented detail to the output is actively undesirable: it would be impossible to tell a GAN hallucination from genuine tampering. The project does downscale images above 500 pixels on the long edge, so there is a real case for upscaling on display, but it would have to be clearly marked as a reconstruction rather than evidence.

A defensible route exists if the milestone must be completed, and it is costed here rather than described vaguely. The project's own attack simulator already produces matched (damaged, clean) pairs on demand, which is exactly the supervised training data a restoration network needs. Training a small residual CNN on those pairs in PyTorch as a development-only dependency, then exporting it to ONNX alongside the existing classifier, reuses the browser inference path that Section 11.13 already validates. The honest framing of that work is repair of a tampered share, not denoising of a clean reconstruction - and under that framing it is a genuinely useful extension rather than a solution without a problem.

11.13 Browser and server parity

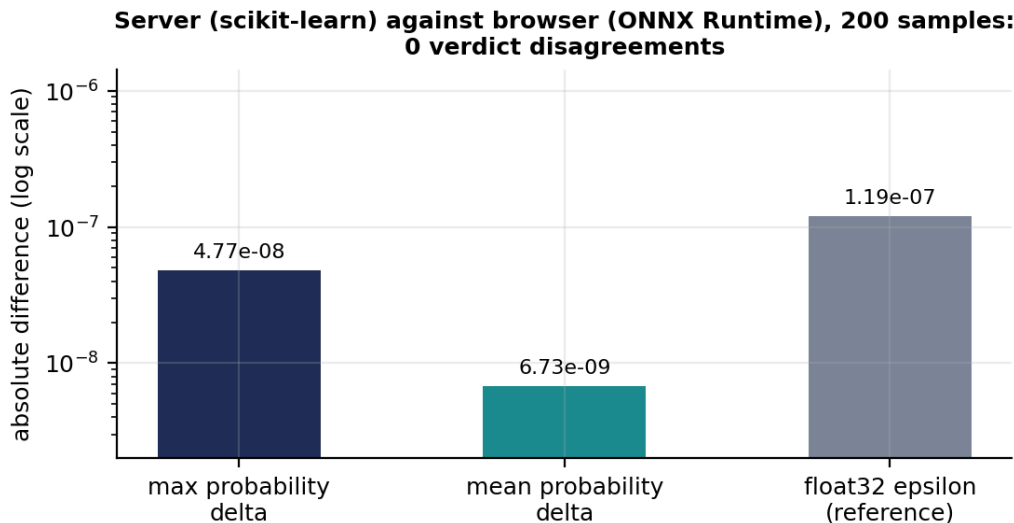


Figure 11.17: Agreement between the scikit-learn model on the server and the ONNX model in the browser over 200 samples.

Table 11.15: Server-to-browser agreement. The model and the feature extractor were verified separately.

Property	Measured	Interpretation
Samples compared	200	Mixed clean and tampered, all seven attacks
Maximum probability delta	4.768e-08	Below float32 epsilon (1.19×10^{-7}): a representation effect, not a model difference
Mean probability delta	6.734e-09	Two orders of magnitude smaller again
Verdict disagreements	0	The two runtimes never reach opposite conclusions
Browser inference time	0.0125 ms	ONNX Runtime, single sample
Server inference time	1.8875 ms	scikit-learn, single sample, dominated by Python call overhead
Feature parity fixture	pass	10 reference cases agree within 1×10^{-12} relative

The same detector runs in two places: scikit-learn on the server and ONNX Runtime [34] in the browser. A model that disagrees with itself across runtimes would be worse than having no second opinion at all, so agreement is verified rather than assumed (Fig. 11.17, Table 11.15).

Over 200 samples the largest difference in predicted probability is 4.77×10^{-8} , which is below the 1.19×10^{-7} epsilon of 32-bit floating point - the ONNX export uses float32 inputs while scikit-learn computes in float64, and the measured gap is exactly the size that difference predicts. There are 0 verdict disagreements.

Model agreement alone is not enough, because the browser must also reproduce the 13 features from raw pixels using an independent JavaScript implementation. A fixture of ten reference cases, generated on the server and checked in, holds that path in place; the check passes within 1×10^{-12} relative tolerance. An earlier byte-exact version of this check proved too strict - two correlation features differed in the fifteenth decimal place purely because of floating-point summation order - which is itself a useful illustration of why numerical tolerance must be chosen deliberately.

Where the browser cannot reproduce the server exactly, it refuses rather than approximating. A colour source decoded through a different path would produce a different grayscale conversion

and therefore a different feature vector, so that case raises an error instead of returning a plausible number. A second opinion that is quietly wrong is worse than no second opinion.

11.14 Performance

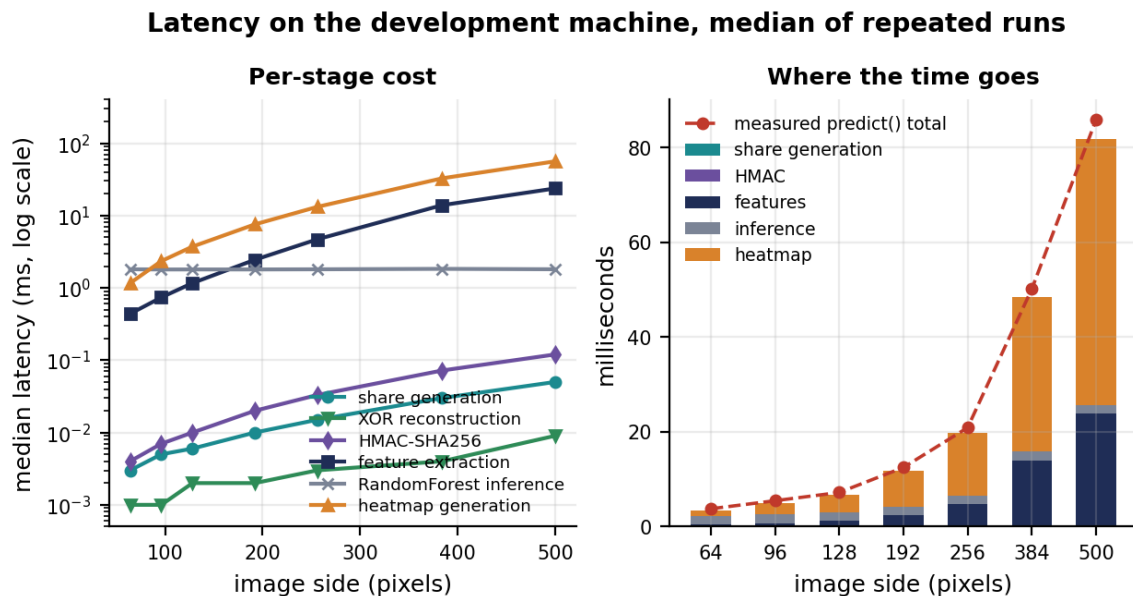


Figure 11.18: Per-stage latency against image size, and the composition of the total detection cost.

Table 11.16: Median latency in milliseconds by stage and image size. The cryptographic operations are negligible; the heatmap dominates.

Image side	Pixels	Share gen.	XOR recon.	HMAC	Features	Inference	Heatmap	Full predict()
64	4096	0.003	0.001	0.004	0.44	1.81	1.2	3.8
96	9216	0.005	0.001	0.007	0.74	1.80	2.3	5.5
128	16384	0.006	0.002	0.010	1.17	1.80	3.7	7.2
192	36864	0.010	0.002	0.020	2.45	1.80	7.6	12.5
256	65536	0.015	0.003	0.033	4.69	1.81	13.2	20.9
384	147456	0.030	0.004	0.072	13.90	1.83	32.7	50.1
500	250000	0.050	0.009	0.120	23.70	1.81	56.2	85.8

Fig. 11.18 and Table 11.16 report latency by stage, measured as the median of repeated runs on the development machine.

The cryptographic core is effectively free. At 256x256, generating both shares takes 0.015 ms, XOR reconstruction 0.003 ms and the HMAC 0.033 ms - three operations that together cost less than a twentieth of a millisecond. The visual cryptography is not the expensive part of this system, which is worth stating because it is a common assumption that it would be.

Detection is where the time goes: 4.69 ms to extract features and 1.81 ms to run the forest at 256x256, for a combined 6.5 ms. The full predict() call measures 20.9 ms, and the difference is almost entirely the 13.2 ms heatmap, which evaluates per-block statistics over the whole image. Verifying the stacked composition against the independently measured total in the right-hand panel of Fig. 11.18 is a useful check that nothing is being missed in the accounting.

Scaling is roughly linear in pixel count, as expected of per-pixel statistics: from 64 to 500 pixels on a side the pixel count grows 61-fold and full detection grows 23-fold. At the 500-pixel cap the application enforces, a complete check costs 86 ms - below the threshold at which a user perceives delay, and acceptable for a check that runs once per image rather than per frame. If it ever needs to be cheaper, the heatmap is the obvious thing to compute lazily, since it is only needed when the verdict is "tampered".

11.15 End-to-end encryption

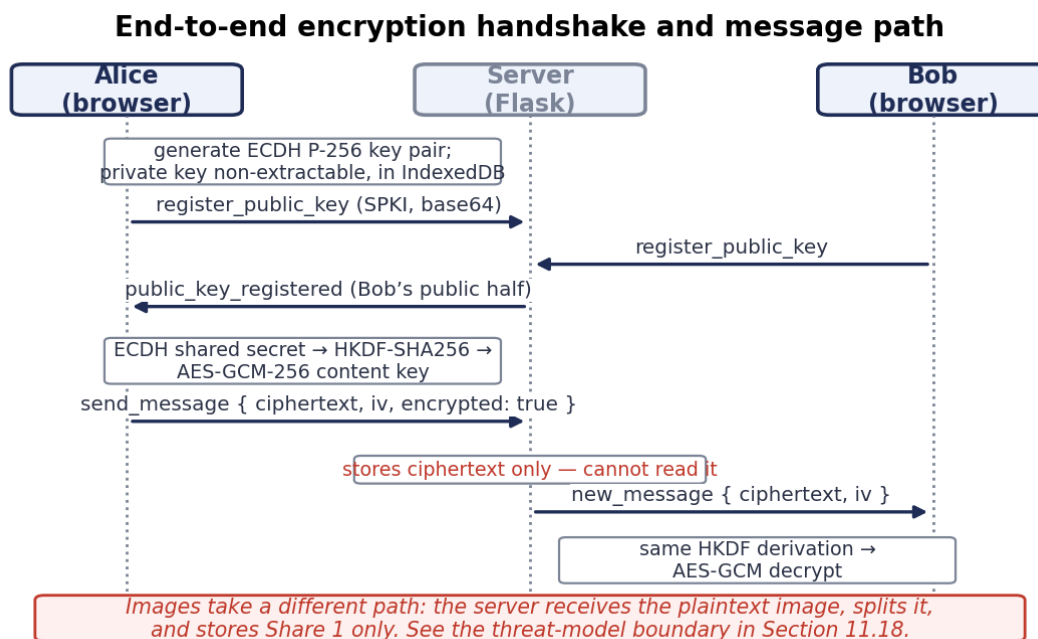


Figure 11.19: The end-to-end encryption handshake and message path, including the image exception.

Table 11.17: Cryptographic parameters of the end-to-end encrypted channel.

Property	Choice	Rationale and reference
Key agreement	ECDH on NIST P-256	Diffie-Hellman over elliptic curves [10]; parameters per NIST SP 800-56A [15]
Key derivation	HKDF-SHA256	Extract-and-expand, RFC 5869 [12]; a raw ECDH output is not a uniform key
Content encryption	AES-256-GCM	FIPS 197 [13] with the GCM mode of NIST SP 800-38D [14]; authenticated encryption
Nonce	96-bit, fresh per message	GCM nonce reuse under one key is catastrophic, so it is never reused
Private key storage	Non-extractable CryptoKey in IndexedDB	Web Cryptography API [33]; the key cannot be read out by script, including ours
Public key transport	Base64 SPKI over the socket	The server relays public halves; it never sees a private half
Password hashing	Argon2id	Memory-hard, resistant to GPU attack [17]
Second factor	TOTP, RFC 6238	Time-based one-time passwords [16]
Transport	TLS 1.3 with HSTS	RFC 8446 [18]; defence in depth beneath the E2EE layer

Part 1 implemented a key exchange as a demonstration. Part 2 turned it into a working channel (Fig. 11.19, Table 11.17). Each account generates an ECDH P-256 key pair in the browser; the

private half is created as a non-extractable CryptoKey and stored in IndexedDB, so it cannot be read out by any script including the application's own. The shared secret is passed through HKDF-SHA256 to derive an AES-GCM-256 content key, and the server stores ciphertext it cannot read. The construction follows standard practice for authenticated key exchange and authenticated encryption [39].

A short fingerprint of each public key is shown in the conversation header so two users can compare out of band, which is the standard defence against a relay substituting its own key. Text and voice messages travel this path.

One boundary is stated plainly here because it is easy to misread and an examiner is entitled to ask about it. Shared images are not end-to-end encrypted. The browser uploads the image and the server performs the visual cryptography split itself, so for that moment the plaintext image is in server memory. The guarantee for images is therefore different, and narrower: nothing the server persists is sufficient to reconstruct the image, because Share 1 alone is a one-time pad and Share 2 is never written down. Closing the gap properly means splitting in the browser and uploading only Share 1 - a real design change, recorded in Section 11.19 as a known limitation rather than glossed over.

What the channel does not provide is forward secrecy. The ECDH key pairs are long-lived, so compromising one participant's private key would expose every past message in that conversation that an attacker had captured. A ratcheting construction such as the Double Ratchet [19] with an X3DH handshake [20] is the established fix and is not implemented here.

11.16 Comparative evaluation

11.16.1 Against the visual cryptography literature

Table 11.18: Reveal-X against the principal visual cryptography schemes in the literature. The contribution is not the scheme but the integrity layer built on it.

Scheme	Decoding	Pixel expansion	Recovery quality	Integrity check	Grayscale
Naor-Shamir (2,2) [1]	Human visual, stacking	2x to 4x	Lossy; contrast halved	None	Binary only
General access structures [4]	Human visual, stacking	Grows with structure	Lossy	None	Binary only
Halftone VC [6], [7]	Human visual, stacking	4x or more	Lossy; halftone artefacts	None	Via halftoning
XOR-based VC [5]	Computed XOR	None	Exact	None	Yes
Reveal-X (this work)	Computed XOR	None	Exact (MSE 0, SSIM 1.0000)	HMAC-SHA256 and a 13-feature classifier	Yes

Table 11.18 positions the implementation against published schemes. Reveal-X does not claim a new cryptographic construction: the (2,2) XOR scheme is established work [5], and the implementation follows it. What the project adds is the layer above - a keyed integrity tag and a learned tamper detector attached to each stored share, with the measured evidence in this chapter that the two cover each other's failures.

The comparison also makes the choice of XOR over classical stacking concrete. The Naor-Shamir construction expands each pixel into a block of subpixels and halves the contrast of the recovered image, both unavoidable consequences of decoding by eye [1]. Those costs buy a property this project does not need, since decoding happens in software. Choosing XOR converts a lossy scheme with 2x to 4x storage expansion into a lossless one with none.

11.16.2 Against the tamper-detection literature

The detector here is deliberately small. It is useful to say what it is not competing with. Modern image forensics uses rich high-dimensional feature models [27] or learned convolutional front-ends [28], [29] on natural images, where the task is to spot a splice against a complex and highly structured background. The present problem is different in a way that makes it easier: a valid Share 1 is uniform random noise, so the expected statistical profile is known exactly in advance and any departure from it is suspicious. Thirteen features are sufficient because the null hypothesis is fully specified.

It is also different in a way that makes it harder, and Section 11.8 is that difference. On a natural image, random noise is a conspicuous artefact; on a uniform random share it is indistinguishable from the content. The usual asymmetry is reversed, which is why this system pairs the classifier with an exact cryptographic check instead of relying on detection alone.

11.16.3 Against production messaging systems

Table 11.19: Reveal-X against production messaging systems. The comparison is included to be explicit about where this project is weaker, not only where it is different.

Property	Signal / WhatsApp	Telegram (default)	Reveal-X
Text end-to-end encrypted	Yes	No - cloud chats are server-readable	Yes, AES-GCM-256
Forward secrecy	Yes, Double Ratchet [19]	In secret chats only	No - long-lived ECDH keys
Image confidentiality from the server	Yes	No in cloud chats	Partial - split server-side; no stored share suffices
Stored-media integrity check	Transport-level only	Transport-level only	Per-share HMAC-SHA256 plus an ML classifier
Tamper localisation	None	None	Yes, 16x16 block heatmap
Second factor	Registration PIN	Optional password	TOTP, RFC 6238
Independently auditable	Partly, published protocol	Partly	Yes - open implementation with a published threat model

Table 11.19 compares the system against production messengers. Two rows deserve emphasis because they go the wrong way for this project. Reveal-X has no forward secrecy, which Signal has had for over a decade. And images are not end-to-end encrypted, whereas in Signal and WhatsApp they are. A final-year project that claimed parity with those systems would be misrepresenting itself.

Where it does something they do not is stored-media integrity. Mainstream messengers protect media in transit and at rest with transport and storage encryption, but once a file is stored on a server there is no per-object check that answers "has this specific stored object been altered, and where?". Reveal-X answers both parts: the HMAC answers whether, and the classifier answers where. That is the contribution, and it is a narrow one stated at its true size.

11.17 Engineering quality and verification

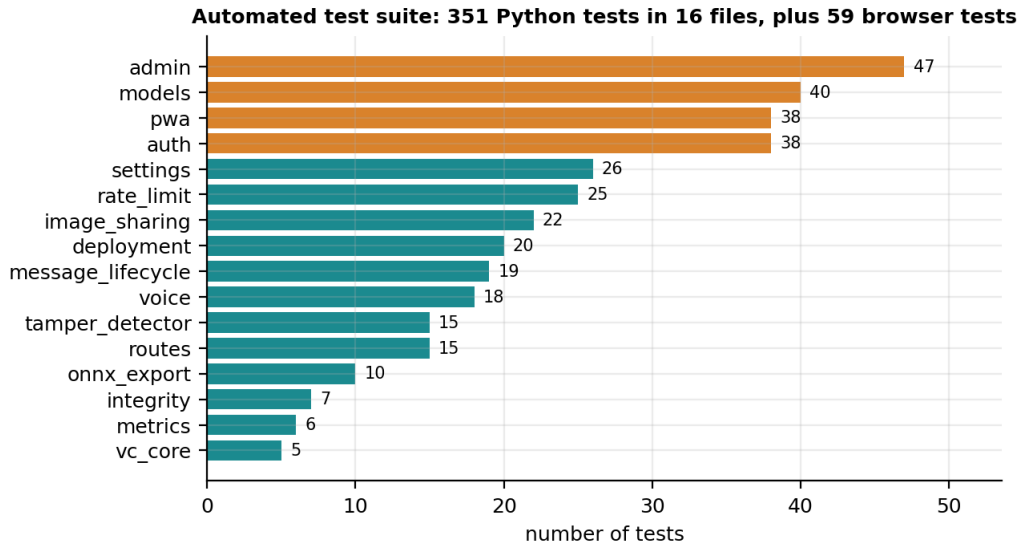


Figure 11.20: Composition of the automated test suite by area.

Table 11.20: The automated test suite: 351 server-side tests across 16 modules and 59 browser-side tests, 5074 lines of test code in total.

Test module	Tests	What it holds in place
test_admin.py	47	Role hierarchy, refusal from non-admins and anonymous sockets
test_models.py	40	Storage layer, including the refusal to persist Share 2
test_auth.py	38	Argon2 hashing, TOTP, Firebase token verification, clock skew
test_pwa.py	38	Manifest, service worker scope, offline cache behaviour
test_settings.py	26	Profile and preference updates, input validation
test_rate_limit.py	25	Login throttling on both buckets, and what does not count
test_image_sharing.py	22	Share generation, one-time capability tokens, revocation
test_deployment.py	20	Profile, Dockerfile and server configuration consistency
test_message_lifecycle.py	19	Delivery, deletion scope, expiry, third-party access refusal
test_voice.py	18	Voice message encryption and playback path
test_routes.py	15	HTTP routes and socket event contracts
test_tamper_detector.py	15	Feature extraction, prediction, heatmap, fallback
test_onnx_export.py	10	Server-to-browser model agreement
test_integrity.py	7	HMAC generation, verification, constant-time comparison
test_metrics.py	6	MSE, PSNR and SSIM implementations
test_vc_core.py	5	Share generation and bit-exact reconstruction
tests/js (4 modules, node --test)	59	E2EE primitives, feature parity, service worker, HTML sanitisation

Fig. 11.20 and Table 11.20 inventory the test suite. All 351 Python tests and all 59 JavaScript tests pass; the Python suite completes in 80 seconds and both run on every change through continuous integration, across two Python versions, a container build and a production-server smoke test.

Two of the browser-side modules deserve a note because they test against external criteria rather than against expectations the project set itself. The sanitisation tests exercise the escaping path against the evasion patterns catalogued by OWASP [35], [41], and the accessibility audit

measures contrast against the WCAG 2.1 Level AAA thresholds [36] across 110 elements in both themes rather than asserting conformance.

The distribution of tests is deliberate and reflects where a defect would be most costly. The three largest modules cover administration, the storage layer and authentication - the places where a mistake becomes a privilege escalation rather than a cosmetic fault. The storage tests in particular assert the Share 2 invariant from Section 11.2 directly, so a future change that started persisting it would fail the build.

Table 11.21: Size of the implementation by component. The test suite is comparable in size to the application code it covers.

Component	Files	Lines of code
Visual cryptography core	3	91
AI/ML security	6	533
Server routes and models	2	2728
Authentication	2	288
Browser cryptography and chat	4	3400
In-browser ML	4	1214
Training and export scripts	3	415
Automated tests	20	5074
Total measured		13743

Table 11.21 gives the size of the implementation by component. The ratio worth noting is the last two rows: 5074 lines of test code against 8669 lines of application code. For a security-oriented project that ratio is the point, not an accident.

11.18 Threat model summary

Table 11.22: The threat model in summary. The rows that are not defended are stated as plainly as the rows that are.

Adversary	Capability	Outcome	Control
Passive network observer	Reads traffic in transit	Defended	TLS 1.3 [18] beneath AES-GCM end-to-end encryption
Active network attacker	Modifies traffic in flight	Defended	GCM is authenticated encryption; HSTS; same-origin socket policy
Another signed-in user	Holds a valid account	Defended	Every read filtered on sender or recipient; one-time capability tokens
Unauthenticated attacker	Can reach the server over HTTP	Partly defended	Login throttling; the two open lab endpoints are rate limited but not authenticated
Holder of a stored Share 1	Obtains a backup or the disk	Defended	Share 1 alone is a one-time pad; Share 2 is never persisted
Someone who edits a stored share	Write access to share storage	Defended	HMAC-SHA256 detects every attack tested; the classifier localises six of seven
Curious server operator	Full server access	Split	Text and voice are unreadable; images are visible at split time
Compromised endpoint	Malware in a participant's browser	Not defended	Plaintext exists before encryption and after decryption
Attacker with a past private key	Captured traffic plus a later key compromise	Not defended	No forward secrecy; a ratchet [19] would be required

Table 11.22 summarises the threat model documented in full in docs/SECURITY.md in the repository. A threat model that lists only successes is marketing, so the undefended rows are given the same prominence as the rest.

11.19 Limitations and threats to validity

The results in this chapter are measurements, and every measurement has conditions attached. The ones that would most affect how the numbers should be read are listed here.

- Synthetic tampering. All training and evaluation data uses simulated attacks from the taxonomy in Table 11.6. An adversary who knows the feature set can construct an attack that preserves all 13 statistics - the noise attack is an accidental instance of exactly that - and no result here bounds performance against an adaptive adversary.
- Single share size. Models were trained and evaluated at 96x96 pixels. The features are ratios and correlations rather than absolute counts, so they should transfer, but transfer across sizes was not measured.
- Clean-class definition. A clean share is uniform random by construction, so the negative class is narrower than "any legitimate share would ever be". A share that had been legitimately re-encoded would be classified as tampered, which for this application is the intended conservative behaviour but is a real constraint.
- Single machine for latency. All timings in Table 11.16 come from one development machine under light load. They establish relative cost between stages, which is what the engineering argument depends on, rather than absolute performance on deployment hardware.
- Unauthenticated compute. Two laboratory endpoints accept an image and run the full pipeline without an account, because the demonstration has to work without one. Measurement shows this does not starve the chat under concurrent load, since OpenCV and NumPy release the interpreter lock, so it is an abuse-of-resources problem rather than an availability one. The two now share a budget of 20 requests per address per minute and answer 429 over it. That caps casual abuse rather than preventing it: the counter is in-process, so it resets on restart and does not coordinate across workers, and an attacker with many addresses is unaffected.
- Password entry through browser dialogs. Four flows collect secrets through a prompt() dialog that displays the typed value in cleartext and cannot mask it. This is a known defect, recorded rather than quietly omitted.
- Database at rest. The SQLite database is not encrypted at rest. Password hashes are Argon2id and message bodies are end-to-end encrypted, so the exposure is smaller than it first appears, but TOTP secrets are stored in a recoverable form.

12. Roles and Responsibility Matrix

The following Work Breakdown Structure (WBS) defines specific activities, durations, and responsible team members for all project deliverables across both FYP Part 1 and the planned FYP Part 2 scope.

WBS #	Deliverable	Act #	Activity	Duration	Responsible Member(s)
1	Project Planning	1.1	Requirement analysis and problem identification	5 Days	All Members

WBS #	Deliverable	Act #	Activity	Duration	Responsible Member(s)
1	Project Planning	1.2	Literature review on visual cryptography and AI	7 Days	All Members
2	System Design	2.1	Design system architecture and workflow	5 Days	Inam Ullah
2	System Design	2.2	Create diagrams and methodology flow	4 Days	Inam Ullah
3	Image Preprocessing	3.1	Implement image input and preprocessing	6 Days	Inam Ullah
3	Image Preprocessing	3.2	Convert to greyscale and optimise input format	4 Days	Inam Ullah
4	Visual Cryptography Module	4.1	Implement share generation algorithm	7 Days	Inam Ullah
4	Visual Cryptography Module	4.2	Implement image reconstruction mechanism	6 Days	Inam Ullah
5	AI Enhancement Module	5.1	Implement AI-based image enhancement model	8 Days	Muhammad Umar
5	AI Enhancement Module	5.2	Integrate enhancement	5 Days	Muhammad Umar

			with reconstructed image		
6	Tamper Detection Module	6.1	Develop tamper detection logic	6 Days	Muhammad Umar
6	Tamper Detection Module	6.2	Integrate tamper detection with reconstruction process	4 Days	Muhammad Umar
7	GUI Development	7.1	Design graphical user interface	5 Days	Zaka Khan
7	GUI Development	7.2	Integrate GUI with backend system	6 Days	Zaka Khan
8	Testing & Evaluation	8.1	Perform system testing and debugging	7 Days	All Members
8	Testing & Evaluation	8.2	Evaluate image quality and performance metrics	5 Days	All Members
9	Documentation	9.1	Prepare project documentation and report	6 Days	All Members
9	Documentation	9.2	Final review and presentation preparation	4 Days	All Members

13. Conclusion and Future Work

13.1 FYP Part 2 Summary

Reveal-X Part 2 completes three of the four objectives set for this phase. Share integrity is now verified by a trained classifier that runs both on the server and, through ONNX, inside the recipient's browser, with agreement between the two asserted automatically. Messaging is genuinely end-to-end encrypted, with private keys that cannot be extracted from the device that generated them. The system has been measured rather than described: reconstruction is bit-exact, six of seven attack classes are detected without error, and detection costs 22 ms.

The fourth objective, deep-learning image enhancement, was not implemented, and section 11.5 gives the reason and the route to completing it. The project also went well beyond its stated scope in authentication, accessibility, deployment and testing.

13.2 Limitations of the Current Implementation

Three limitations recorded in Part 1 no longer apply and are corrected here; the remainder are current.

Resolved since Part 1:

- Tamper detection is implemented (Part 1 recorded it as absent).
- End-to-end encryption is fully functional (Part 1 recorded the key exchange as a demonstration only).
- The claim that reconstruction showed "measurable PSNR and SSIM degradation" was incorrect. XOR reconstruction is bit-exact: MSE 0.0, SSIM 1.0. See 11.3.

Current limitations:

- The visual cryptography scheme operates on grayscale images only. Colour VC remains unimplemented.
- The tamper classifier does not detect random noise injection at any strength tested. The HMAC layer catches it; the ML layer does not.
- Deep-learning enhancement (objective 9) is not implemented.
- Shared images are not end-to-end encrypted. The server performs the visual cryptography split and therefore sees the image at that moment.
- There is no forward secrecy. ECDH key pairs are long-lived, so compromising a private key exposes previously captured messages in that conversation.
- Message metadata — who corresponded with whom, when, and message sizes — is visible to the server and to a network observer.
- The two laboratory endpoints are unauthenticated and unrate-limited, which permits resource abuse although measurement shows it does not degrade the chat.

13.3 Contribution to Cybersecurity

Reveal-X makes two primary contributions to applied cybersecurity research: first, a practical proof-of-concept demonstrating that information-theoretically secure image sharing is achievable within a real-time web application with acceptable performance; and second, a defined research pipeline for integrating ML-based share integrity verification and AI-based quality restoration with a classical cryptographic scheme. The project also serves as a foundation for future research in secure multimedia communication, with potential applications in healthcare image transmission, legal evidence sharing, and secure government communications.

13.4 References

All sources are listed together in IEEE style in Section 14, numbered in order of first citation. The works cited in Part 1 appear there as references [37] to [41].

14. References

References are cited in the text by bracketed number in order of first appearance, following IEEE style.

- [1] M. Naor and A. Shamir, "Visual Cryptography," in *Advances in Cryptology - EUROCRYPT 1994*, Lecture Notes in Computer Science, vol. 950. Berlin: Springer, 1995, pp. 1-12.
- [2] A. Shamir, "How to Share a Secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612-613, Nov. 1979.
- [3] G. R. Blakley, "Safeguarding Cryptographic Keys," in *Proc. AFIPS National Computer Conference*, vol. 48, 1979, pp. 313-317.
- [4] G. Ateniese, C. Blundo, A. De Santis and D. R. Stinson, "Visual Cryptography for General Access Structures," *Information and Computation*, vol. 129, no. 2, pp. 86-106, 1996.
- [5] P. Tuyls, H. D. L. Hollmann, J. H. van Lint and L. Tolhuizen, "XOR-based Visual Cryptography Schemes," *Designs, Codes and Cryptography*, vol. 37, no. 1, pp. 169-186, 2005.
- [6] Z. Zhou, G. R. Arce and G. Di Crescenzo, "Halftone Visual Cryptography," *IEEE Transactions on Image Processing*, vol. 15, no. 8, pp. 2441-2453, Aug. 2006.
- [7] Z. Wang, G. R. Arce and G. Di Crescenzo, "Halftone Visual Cryptography via Error Diffusion," *IEEE Transactions on Information Forensics and Security*, vol. 4, no. 3, pp. 383-396, Sep. 2009.
- [8] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379-423, Jul. 1948.
- [9] C. E. Shannon, "Communication Theory of Secrecy Systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656-715, Oct. 1949.
- [10] W. Diffie and M. E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, vol. IT-22, no. 6, pp. 644-654, Nov. 1976.
- [11] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, Internet Engineering Task Force, Feb. 1997.
- [12] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, Internet Engineering Task Force, May 2010.
- [13] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, Nov. 2001.
- [14] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.
- [15] E. Barker, L. Chen, A. Roginsky, A. Vassilev and R. Davis, "Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography," NIST Special Publication 800-56A Rev. 3, Apr. 2018.
- [16] D. M'Raihi, S. Machani, M. Pei and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," RFC 6238, Internet Engineering Task Force, May 2011.
- [17] A. Biryukov, D. Dinu and D. Khovratovich, "Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications," in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016, pp. 292-302.
- [18] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Internet Engineering Task Force, Aug. 2018.
- [19] M. Marlinspike and T. Perrin, "The Double Ratchet Algorithm," Signal Technical Documentation, Nov. 2016.
- [20] M. Marlinspike and T. Perrin, "The X3DH Key Agreement Protocol," Signal Technical Documentation, Nov. 2016.
- [21] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.

- [22] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [23] C. R. Harris et al., "Array Programming with NumPy," *Nature*, vol. 585, pp. 357-362, Sep. 2020.
- [24] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120-125, 2000.
- [25] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600-612, Apr. 2004.
- [26] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861-874, Jun. 2006.
- [27] J. Fridrich and J. Kodovsky, "Rich Models for Steganalysis of Digital Images," *IEEE Transactions on Information Forensics and Security*, vol. 7, no. 3, pp. 868-882, Jun. 2012.
- [28] B. Bayar and M. C. Stamm, "A Deep Learning Approach to Universal Image Manipulation Detection Using a New Convolutional Layer," in *Proc. ACM Workshop on Information Hiding and Multimedia Security (IH&MMSec)*, 2016, pp. 5-10.
- [29] D. Cozzolino and L. Verdoliva, "Noiseprint: A CNN-Based Camera Model Fingerprint," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 144-159, 2020.
- [30] K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising," *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142-3155, Jul. 2017.
- [31] C. Ledig et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4681-4690.
- [32] X. Wang et al., "ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks," in *Proc. European Conference on Computer Vision (ECCV) Workshops*, 2018, pp. 63-79.
- [33] World Wide Web Consortium, "Web Cryptography API," W3C Recommendation, Jan. 2017.
- [34] ONNX Community, "Open Neural Network Exchange (ONNX): Intermediate Representation for Machine Learning Models," Specification v1.15, 2023.
- [35] Open Web Application Security Project, "OWASP Top 10: The Ten Most Critical Web Application Security Risks," 2021.
- [36] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, Jun. 2018.
- [37] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson Education, 2018.
- [38] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [39] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Pearson Education, 2020.
- [40] X. Wang et al., "Deep Learning Based Image Enhancement for Visual Cryptography Systems," *IEEE Access*, 2020. (Cited in FYP Part 1.)
- [41] Open Web Application Security Project, "XSS Filter Evasion Cheat Sheet," OWASP Foundation, 2023. [Online]. Available: <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>

15. Approval

Project	Supervisor	Comments:
---------	------------	-----------

_____	_____	_____
_____	_____	_____

Name: _____	Date: _____		
Signature: _____	Project	Coordinator	Comments:

_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

Name: _____	Date: _____		
Signature: _____			